SCHOOL OF COMPUTER SCIENCE · UPES DEHRADUN

SpeakQL

Frontend Specification

Every screen, every workflow and every state the interface can be in — including the states where nothing has arrived yet and the ones where the request failed — with a screenshot of each taken from the running prototype, and a test matrix covering all of them. Read this before writing frontend code, and again when running the test pass before the review.

Interface specification and test plan. Aligned to Backend Plan Revision 5.

Minor Project · MCA (AI & ML)
Garv Bhargava · Dhruv Chhatrawal · Kritant Poudnel
Mentor: Mr. Pankaj Dadure

1. What this document is

The Backend Plan says what the server does. This says what a person sees, in what order, and what must be true afterwards. The screenshots are not mock-ups: they are captures of `speakql-prototype.html`, which implements every captured workflow here with scripted data and no server.

PART	WHAT IT COVERS
§2–4	The design system, and the two applications the product actually is
§5–19	Fifteen workflows, each with the screens it passes through and what must be true at every step
§20	Accessibility and input — keyboard, focus, assistive technology, and the responsive floor
§21	The demonstration script — the order to show things in, on the day
§22	The test matrix: every workflow and every refusal, as numbered cases
§23	What is automated today, and what a person still has to check by eye
§24	Build notes for the React version

> **One rule governs the whole document.**
> The interface is never the enforcement point. Every screen described here hides things a person may not have, but the server has already refused them. If a screen is the only thing stopping an action, that is a bug in the backend, not a feature of the frontend.

1.1 A convention about the figures

Sections carrying a figure are captured from the working prototype. Three sections — §11, §13 and the stale-merge half of §16 — specify states the prototype does not implement yet; they are marked **No capture yet** and are built in weeks 6 to 8. A specification that only described what already exists would be a description, not a plan.

Two existing captures carry a known debt, recorded here rather than left to be noticed in the review: **Figures 15 and 23 were taken mid-scroll**, so the seal band and the headline figure — which the surrounding text describes — sit above the top of the frame. Both are specified in full in §10.1 and §14, and both captures are to be retaken from the top of the answer before submission. This is listed as a task in §24.

2. The interface in one paragraph

A person signs in with a code sent to their work email. If nobody from their company has ever signed in, they are asked one question — are you setting this up, or joining — and never asked again. An owner lands on their organisation; a member lands on a question box. They type a question in plain English and watch the system name each step as it runs, then get back a number, a chart, an explanation and the SQL that produced it — or, in Readout mode, the number and the rows behind it. If the answer is short because data is missing, the missing cells are marked and can be filled where they stand: an owner's correction lands in the real table and the answer recalculates immediately, a member's goes to the owner to approve, and if the row moved in the meantime the merge is refused rather than applied. Nothing that is not a single read-only SELECT ever runs, and when something is refused the interface says which rule refused it.

3. The design system

3.1 Colour — one token set, two grounds

The console is **dark by default**. An analyst holds this screen for an hour at a stretch reading figures off it, and on a dark ground the only things carrying light are the number, its status, and whatever the system is working on at that moment. The porcelain console is kept in full, but it is now the secondary theme.

These are not two designs. There is **one set of token names, valued twice**. No rule anywhere in the stylesheet names a colour — every one of them names a token — so the whole product turns over on the two blocks below, and no component knows which ground it is standing on. Where a value genuinely had to know which way the ground faces, that dependency is itself a token, listed in the second table, rather than a special case buried somewhere in a rule.

Colour is never decorative. It is reserved entirely for meaning, so that a person can tell at a glance whether they are looking at a verified number, a proposed one, or a refusal — without reading a word.

TOKEN	DARK (DEFAULT)	LIGHT	MEANS
paper / panel / sunk	`#0A0E10` `#111719` `#161D20`	`#EDEEEC` `#FFFFFF` `#F5F6F4`	Ground, surfaces, the recessed surface. On a dark ground the recessed one is the **lightest** of the three, because depth there reads as lift rather than as shadow
ink / ink-2 / ink-3	`#E9EFEE` `#9FACAA` `#687674`	`#14181A` `#4C5254` `#7F8683`	Primary, secondary and muted text
rule / rule-2	`#212A2D` `#2C383B`	`#DEE1DE` `#C7CBC7`	Hairlines and borders
verified	`#3FD68B`	`#0E6B4E`	Real, accepted data. A merged correction. A passing check
pending	`#A78BFA`	`#6242BE`	A **value** proposed but not yet accepted
awaiting	`#E3A55C`	`#8F5400`	A **person** who has not acted yet: a merge request in someone's queue, a transcript awaiting confirmation. Distinct from `pending`, which is about the number rather than the wait
blocked	`#FF6E6E`	`#AF241C`	The validator refused it
clarify	`#5CA8F2`	`#1C5C9E`	The system is asking back
refusal	`#7C8886`	`#5A615E`	The data cannot answer it — deliberately colourless
caution	`#E3A55C`	`#8F5400`	Low confidence, or an answer short of data
working	`#4A9EFF`	`#1C5C9E`	A step in progress. The one token whose reasoning changed; see below
series 1–3	`#57A9FF` `#FF9A5C` `#42D9A0`	`#2a78d6` `#eb6834` `#1baf7a`	Chart series only

`awaiting` shares its value with `caution` in both themes. That is deliberate: both mean "something here is unfinished, and it is not an error". They stay separate tokens because they will not necessarily stay the same colour, and a card waiting on a person is a different fact from a number short of data.

`working` is the one token whose reasoning changed, and it is worth saying so plainly, because the previous revision argued the opposite case. It used to be ink-2, on the argument that work in progress is not a status and colouring it would imply one. On a dark ground that argument fails in practice: ink-2 is also the colour of every muted label on the screen, so a step that is *running* becomes indistinguishable from one that has not started — which is the single thing the stage ladder exists to show. It is now a blue accent, and the only place blue appears outside a chart. It still does not assert a status. It means *here, now*, and it goes out the moment the step lands.

3.2 The tokens that had to know which way the ground faces

Five values cannot be derived from the table above, because their correct answer depends on whether the ground is dark or light. Each is a token in its own right, so the dependency is declared once instead of being rediscovered in every rule that hits it.

DERIVED TOKEN	WHAT IT RESOLVES	WHY IT CANNOT BE INFERRED
solid / on-solid	The fill of a primary button or a filled chip, and the text that sits on that fill	In light this is ink on white; in dark it inverts to near-white with dark text. A rule that wrote `#FFFFFF` for the label would be wrong in exactly one of the two themes, silently
ring / focus-edge	The focus halo, and the border a focused field takes	Ink is the right focus border on porcelain and much too loud on a dark ground, where the field would flare brighter than the value being typed into it. Dark uses the `working` accent instead
tint / tint-2 / tint-edge	How much of a status colour a background wash or a border takes	A 7% wash that reads clearly against white disappears against `#0A0E10`. The dark theme carries roughly double, and the whole system tunes from these three numbers
bar-* (six values)	The walkthrough bar at the foot of the screen, and the toast	The one surface that was a deliberate inversion. On a dark ground it stops inverting and becomes a raised surface, because a near-white slab across the bottom of a dark console is not emphasis — it is a lamp
lift / glow / sweep	The shadow under a floating card, the halo on whatever is working, and the light that crosses a busy bar	Porcelain separates surfaces with shadow; a dark ground separates them with light. The same component needs opposite treatments and should not have to ask

Both palettes were then measured rather than eyeballed, under one stated method: WCAG 2.1 for text contrast, CIEDE2000 for the separation between chart series, and Machado, Oliveira and Fernandes (2009) at severity 1.0 to simulate the three dichromacies. Revision 5 quoted figures from a different metric, so its numbers are not comparable with these; both palettes were re-run together under the one method, which is the point.

MEASUREMENT	DARK	LIGHT	FLOOR
Body text (ink) on its own ground	16.7 : 1	15.4 : 1	4.5 : 1
Secondary text (ink-2), worst of the three surfaces	7.3 : 1	6.8 : 1	4.5 : 1
Muted labels (ink-3), worst of the three surfaces	3.6 : 1	3.2 : 1	3 : 1 — non-essential labels only
Weakest status token, worst of the three surfaces	6.2 : 1	5.2 : 1	4.5 : 1
Chart series against the surface behind them	7.4 to 10.1 : 1	2.8 to 4.4 : 1	3 : 1
Worst series pair, normal vision (CIEDE2000)	42.1	44.8	15
Worst series pair, worst of the three dichromacies (CIEDE2000)	12.3	15.9	8

Two things follow, and the second is the more important. The dark chart series clear 7:1 against the surface behind them where the porcelain series sat between 2.8 and 4.4 and one of the three fell below 3:1 — so on charts specifically, the dark theme is the more legible of the two. But the rule that **every bar carries its own value label** is kept regardless: the worst pair separates by 42 in normal vision and by 12.3 under tritanopia, and a margin that collapses that far is not one to spend on colour alone.

> **No rule in this stylesheet names a colour.**
>
> The two token blocks are the only place a hex value is allowed to appear, which is what makes switching themes a thirty-line change rather than an audit — and it is checkable, because searching the stylesheet for a hex literal outside those blocks should return nothing. It did not, the first time the dark theme was built: about twenty rules across a dozen components had written `#FFFFFF` where they meant *the text that sits on a filled surface*, and the walkthrough bar had assumed that filling itself with ink would read as emphasis on any ground. Each of those became one of the five derived tokens above. The check is worth more than the fix.

The three specification documents, this one included, are set in the dark theme — the same token set, read by the same names, from `docs-src/speakql_doc.py`. There is no second palette anywhere in the toolchain: the page background, the table fills, the rules and the callout edges all resolve from the fourteen values above, which is why the documents and the product cannot drift apart.

The trade is stated rather than hidden. A dark page needs a printer willing to lay down a full-bleed dark ground and it uses a great deal of ink, so these are documents meant to be **read on a screen**. Setting `SPEAKQL_THEME=light` before a build re-runs the identical source against the porcelain values and produces the printable set — which is itself the demonstration that the theme is one block of values and not a rewrite.

3.3 Type

ROLE	FACE	WHY
Interface text	Instrument Sans	Contemporary, tight, not one of the defaults every dashboard reaches for
Numbers, SQL, labels, identifiers	IBM Plex Mono	A headline figure is set as data, not as display type — which is what the product is about

3.4 Layout

A two-column console: a fixed 246px navigator, and a stage that fills the rest. The stage carries a 52px top bar, then one screen at a time. On the Ask screen the stage splits vertically into a scrolling conversation and a fixed composer. Corners are 3–4px, never more. Elevation is carried by hairlines rather than shadows, with two exceptions that float above the page rather than sitting in it: the toast, and the pipeline card on the sign-in screen.

The sign-in screen is the one two-column exception, and it is split down the middle: the form on the left, and on the right the twelve-step pipeline of §6 running by itself, at a third scale, on a loop. It is scripted, but it is scripted from the real thing — the same five surfaces in the same order, the same 0.55 routing threshold, the same four validation layers — and one run in three is **refused by the validator rather than answered**. A front door that only ever demonstrates the happy path makes a promise the product then has to break. The panel is `aria-hidden`, carries no control, and everything it says is said again inside the product, so nothing is lost when it is dropped below 1080px.

The layout is desktop-first and says so. Its responsive floor and what happens below it are specified in §20.4 rather than left to the browser.

3.5 Component inventory

Each component's anatomy is given as height, internal padding and the elements it must contain, because a state list alone is not enough to build from.

COMPONENT	ANATOMY	STATES
Product role NEW	Plain text in the table cell, ink-2. **Not a chip.** A filled chip here read as a button somebody was supposed to press, and a product role is a fact about a person, not an action. The per-database grants beside it stay chips, because those are states	Owner, Member
Chip	20px tall, 6px horizontal padding, 3px radius, 11px mono uppercase. Optional 5px leading dot	neutral, solid, analyst, viewer, verified, pending, awaiting, blocked, caution, clarify
Segmented control	28px tall, 1px hairline, equal-width segments, pressed segment inverts to ink	two or three options, one pressed, options can be disabled
Seal band	32px tall, sunk ground, full answer width, hairline beneath. Carries route · confidence · latency · rows, right-aligned, mono	answer, blocked, clarify, refusal, low confidence, incomplete
Stage ladder NEW	12 rows, 18px each, mono step name left, state glyph right. Collapses to a single summary row on completion	waiting, working, done, skipped, failed
Strip	Full width, 10px padding, 2px leading edge in the tone colour, one line of text plus optional action	neutral, caution, blocked, pending, awaiting, refusal
Gap cell	Matches its table row height. Empty state shows a 12px dashed underline, not a placeholder character	read-only gap, fillable gap, input, filled, pending
Layer ladder	4 rows, 22px each, layer name and what it catches	pass, fail, one marked as the layer that caught it
Check list	4 rows, 18px each, glyph plus rule name	pass, fail, awaiting
Merge card	Header row with chip and identity, two-column body of real against proposed, note row, action row. **Stale variant adds a third column**	awaiting, merged, rejected, stale
Data table	Header 30px, rows 30px, mono numerics right-aligned, hairline between rows	plain, editable, pending row, filled row, truncated
Stat tile	86px tall, value in 24px mono, label in 11px, optional footnote	value, label, footnote
Retry strip NEW	Full width, 10px padding, message plus one action button. Never more than one action	timeout, offline, server error, rate limited, session expired

4. Two applications, not one

This is the most important structural decision in the frontend. SpeakQL is not one application with things hidden; it is two applications that share a component library. An owner and a member have different home screens, different navigation and different vocabulary.

	OWNER	MEMBER
Home	Their organisation	The question box
Navigation	Organisation, Databases, People and access, Merge requests, Ask	Ask, Fill a gap, Your activity, What you can reach
Navigation in a **personal workspace** NEW	Workspace, Databases, Ask. **People and access and Merge requests are not shown at all** — see below	Not applicable; a personal workspace has no members
Correcting data	Writes the real table, immediately	Proposes; the owner merges
Their own pending values	None — an owner has nothing pending, ever	Appear in their own answers, labelled, and nobody else's

	OWNER	MEMBER
Sees other people's questions	Yes, their organisation only	No — own activity only
Sees another company	No. Nobody can	No

4.1 A personal workspace is a third shape, not a smaller owner

New in this revision. A free-mail signup owns an organisation of exactly one person (§6.1). The navigator used to show them the full owner set, which meant two screens that could never do anything: **People and access**, where there is nobody to grant anything to, and **Merge requests**, where no merge can exist because a merge is raised by a member and there are none. Both are now absent from the navigator, along with the "Data quality" heading above them.

- The **workspace row carries the database count**, because in a workspace of one that is the only number on the screen that means anything.
- The header line reads *"N databases · just you"* rather than counting people.
- The organisation screen shows **three tiles, not four** — databases, questions asked, served locally. A tile whose value can only ever be zero is not information.
- The isolation note that used to live on the People screen **moves onto the workspace screen**, because the screen it was standing on is gone and the explanation is the more important half.
- The merge-digest policy strip is hidden, for the same reason the queue is.

> **Absence is the feature here, and it is worth saying out loud.**
>
> A greyed-out "People and access" would invite the question *how do I add someone?*, and the honest answer is that domain resolution can never match this workspace, so nobody can arrive by signing up. Removing the screen states that; disabling it only hints at it. The same reasoning produced the dead-end screen in §6.3 rather than a disabled button.

4.2 What the frontend must never do

- **Never decide a role.** It arrives in the session from the server and is displayed, never computed.
- **Never hide SQL as a security measure.** In Readout mode the server has already removed it from the response body. The frontend hides nothing that matters.
- **Never construct a database identifier from user input.** Connection identity comes from the session.
- **Never show a number without saying where it came from.** Every answer carries its seal: route, confidence, latency, row count — and whether it used the asker's own pending value.
- **Never present a refusal as an answer.** The five non-plain-answer modes each look visibly different from a plain answer and from each other.
- **Never present a failure as a refusal.** A timeout, an offline browser and a validator rejection are three different things, and §13 gives each its own treatment. Showing "no answer" for all three teaches people to distrust the refusals that are real.

5. Workflow A — registering a company

The first person from a domain nobody has registered. This is the only moment in the product where somebody chooses their own permissions, so it is worth walking slowly.

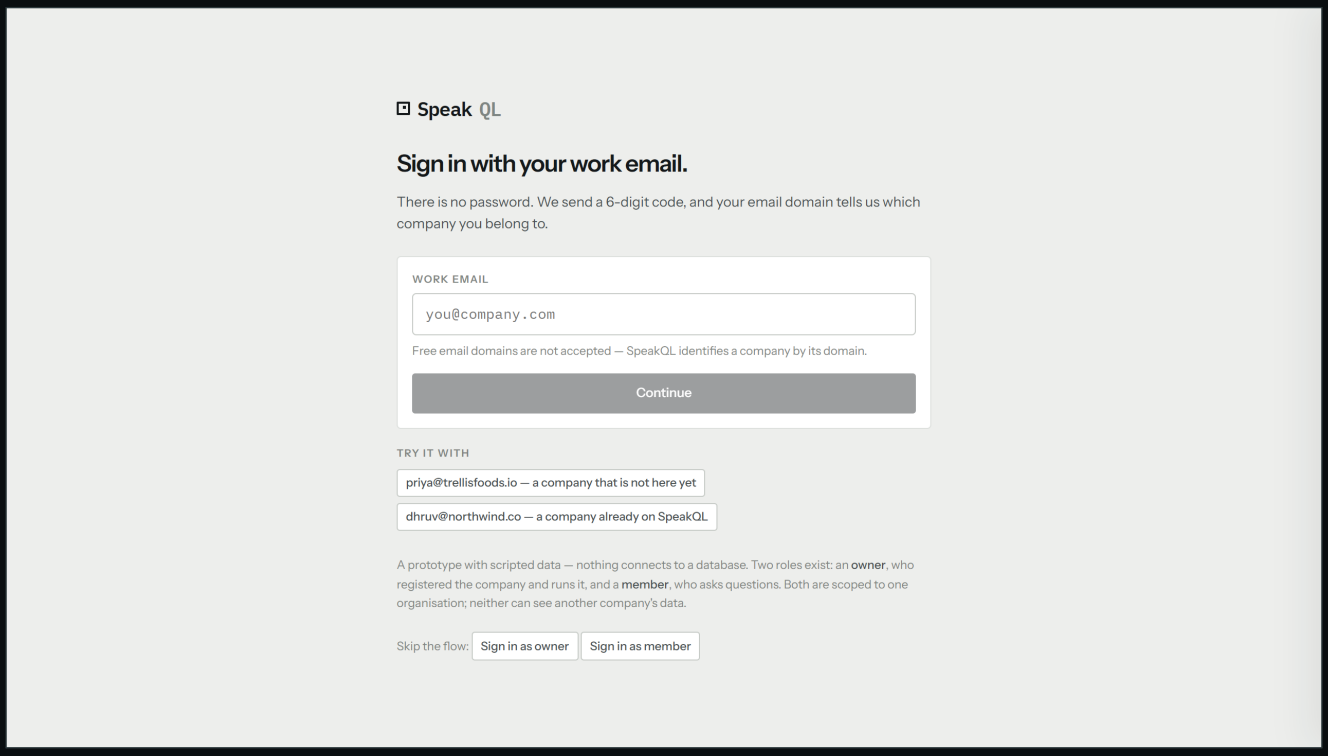


Figure 1. The entry point. One field. No password anywhere in the product, and the hint under the field explains why the domain matters before the person has typed anything.

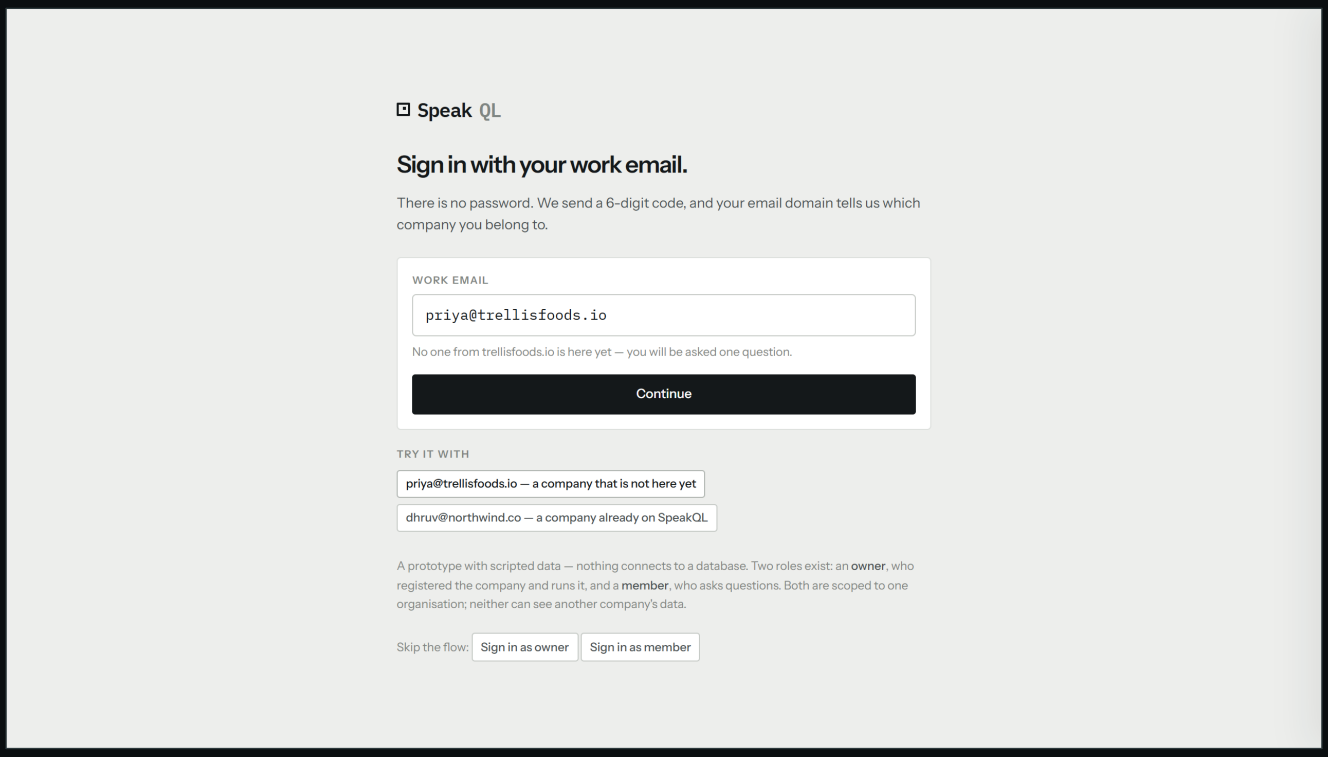


Figure 2. As soon as the address is valid, the hint tells them what is about to happen: nobody from this domain is here yet, so one question is coming. Nothing is a surprise.

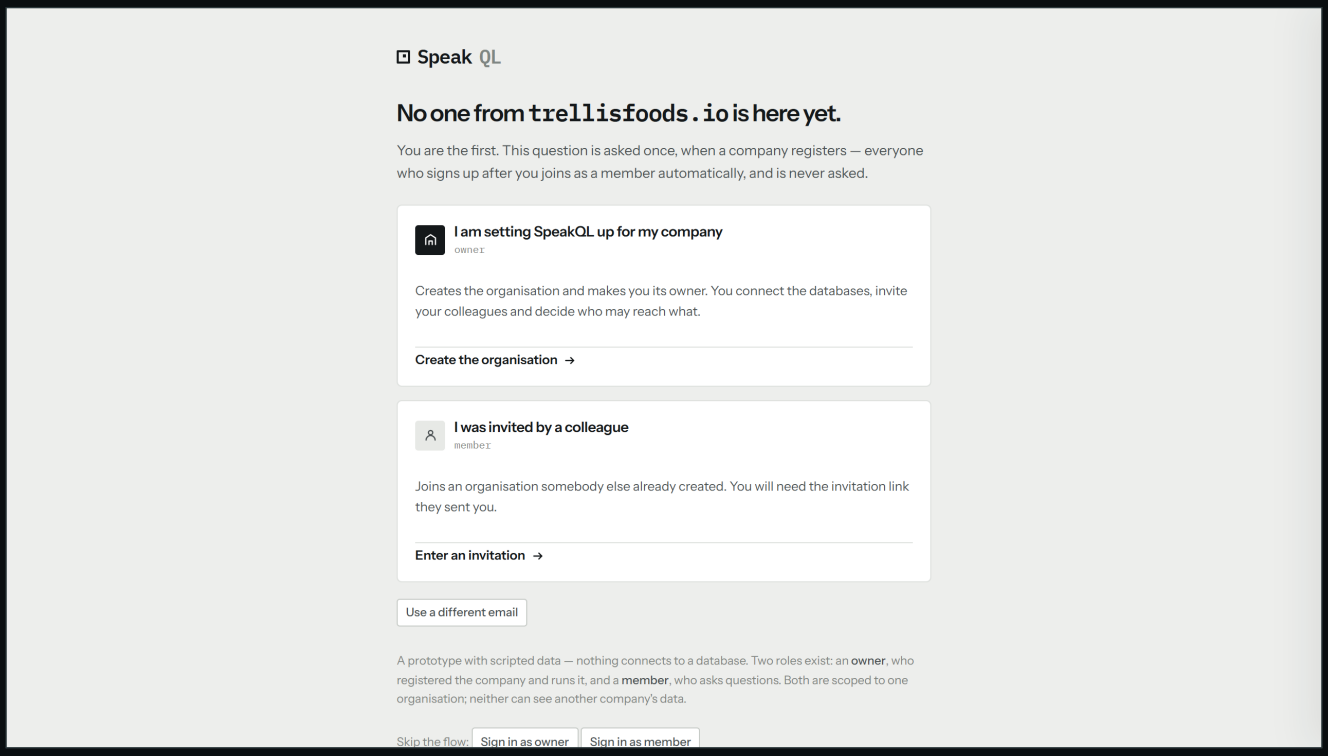


Figure 3. The question, asked once. The heading names the domain so it cannot be mistaken for a generic prompt, and the body states plainly that everyone after them joins as a member and is never asked this.

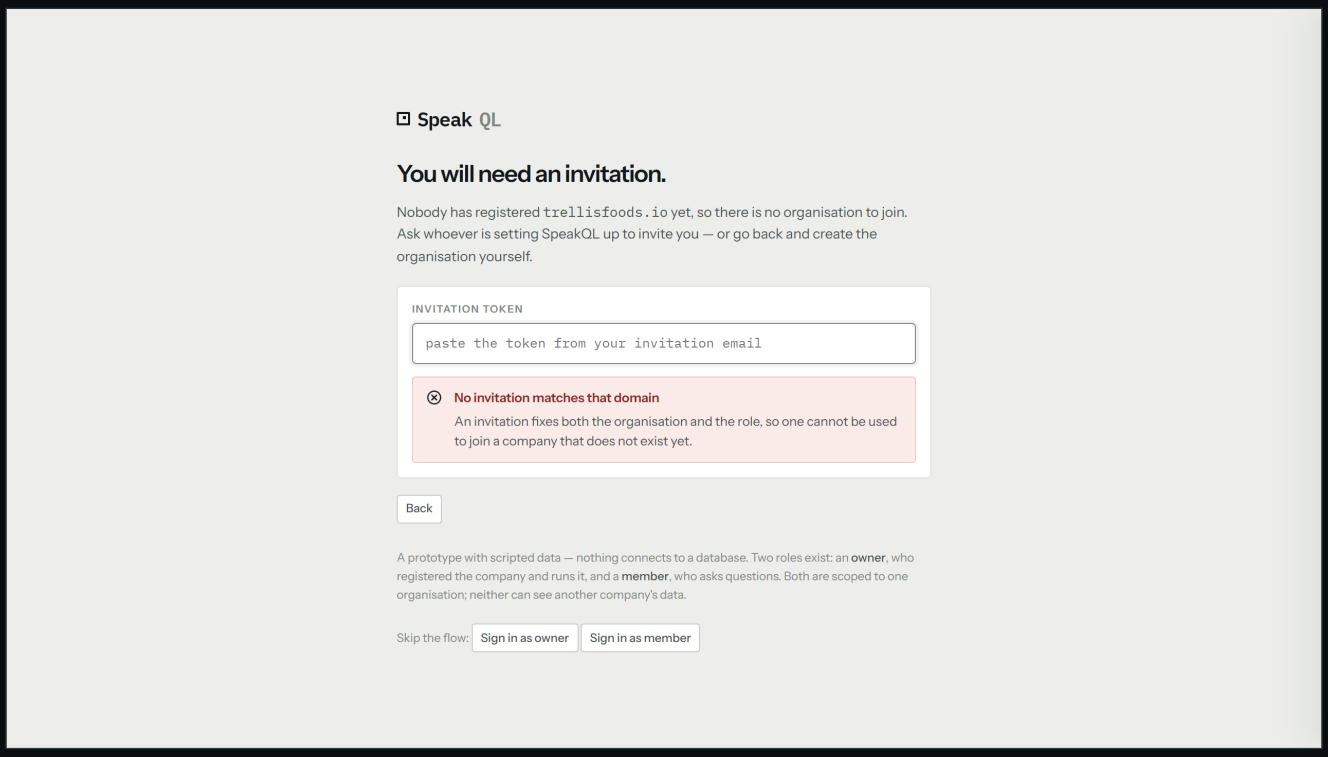


Figure 4. Choosing "I was invited" on a domain nobody has registered. The interface does not pretend this can work: there is no organisation to join, and an invitation is issued by an organisation. The honest end of the branch, not a dead button.

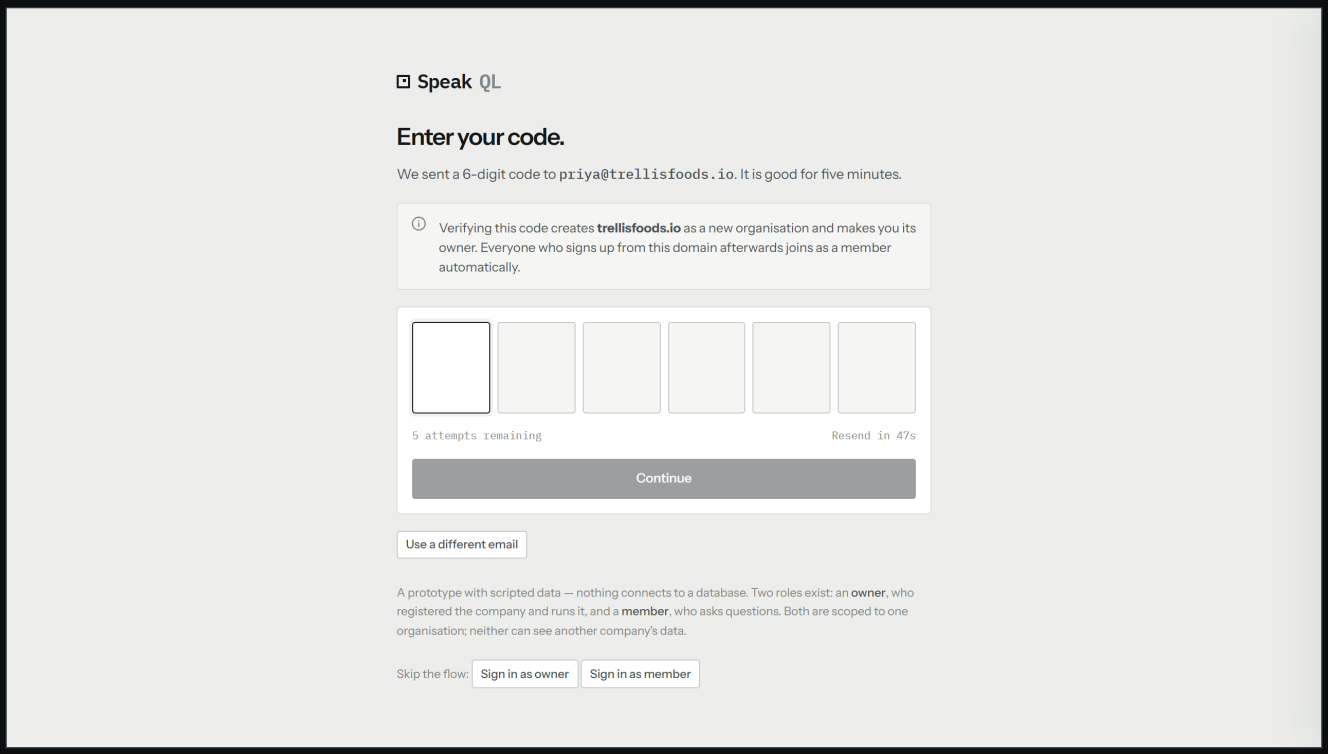


Figure 5. Choosing owner. The code screen states the consequence before they commit: verifying creates the organisation and makes them its owner.

STEP	PERSON DOES	INTERFACE SHOWS	SERVER DOES
1	Types an email	Live hint naming the company, that the domain is new, or that this will be a personal workspace	Resolves the domain — company domains only
2	Continues	The one question, naming the domain	Returns `first_user: true`
3	Picks owner	Code screen, stating that this creates the organisation	Sends a 6-digit code
4	Enters the code	Digits fill left to right, Continue enables on the sixth	Creates the organisation and the owner, mints a session

6. Workflow B — joining a company already here

The same screen, a different path, and the difference is the whole point of the signup design.

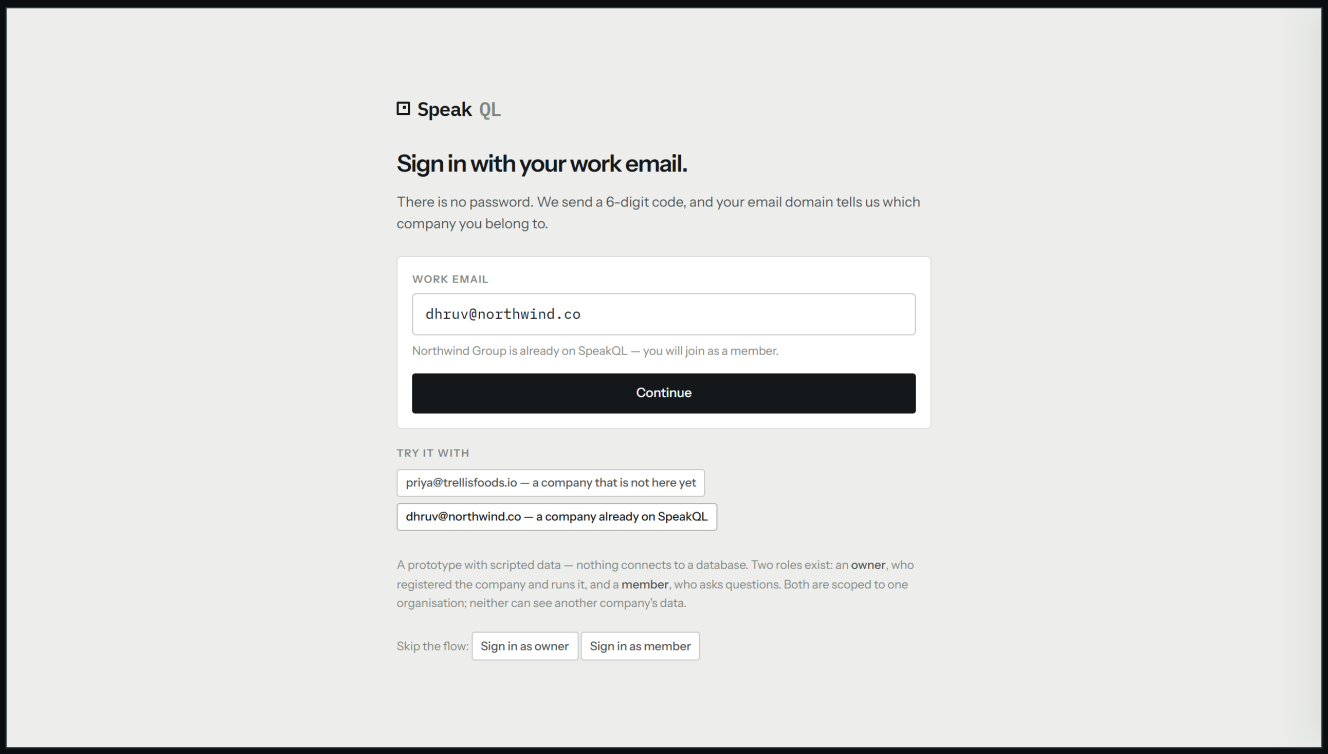


Figure 6. A known domain. The hint names the company and says what they will be. No question is coming.

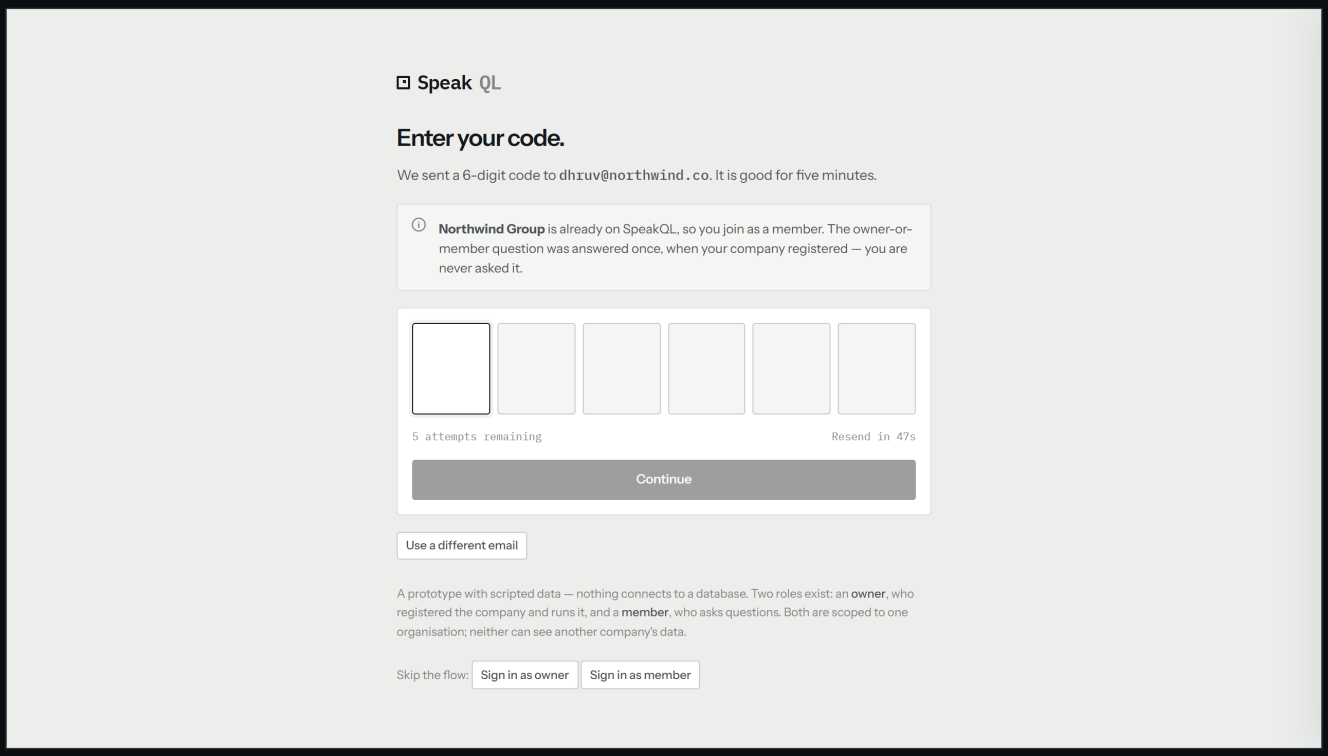


Figure 7. Straight to the code. The strip explains why they were not asked: the question was answered once, when their company registered. A person who has heard a colleague mention "it asked me if I was the owner" now knows why it did not ask them.

> **The behaviour to test hardest.**
>
> These two paths differ by exactly one thing — whether the domain is known — and getting it wrong in either direction is serious. A second owner created by accident is a privilege escalation; a first user forced to be a member leaves a company with no owner and no way to get one.

6.1 The third path — a personal address

A free-mail address takes neither branch above. It gets a **personal workspace**: an organisation of one, with no domain, created without a single question. Backend Plan §7.2 is the rule that makes this safe, and the interface's job is to make the difference legible rather than to enforce it.

WHAT THE PERSON SEES	WHY IT IS WORDED THAT WAY
The hint reads *"gmail.com is not a company, so this will be your own personal workspace — just you, and nobody else can join it by email domain."*	It states the consequence before they commit, exactly as the owner branch does. A person who expected to join their team needs to find out here, not afterwards.
No owner-or-member question is shown.	There is nothing to ask. They are the only person in it, so they are its owner by construction.
The organisation name is their address, and the sidebar chip reads `Personal`.	A workspace with no domain should not pretend to be a company. The chip is the one visible difference between the two kinds anywhere in the product.
An invitation banner sits on the People screen: *"Nobody can join by email domain. Invite them by name."*	This is the only route in, so the screen that would otherwise look broken says why it looks that way.

> **What the interface must never imply here.**
>
> That two people with Gmail addresses are in the same place. They are two separate organisations and cannot see each other, so no screen may ever suggest a shared "Gmail" space — no shared people list, no discoverable workspaces, no "others from your domain". The absence of those is the feature.

7. Workflow C — the owner's home

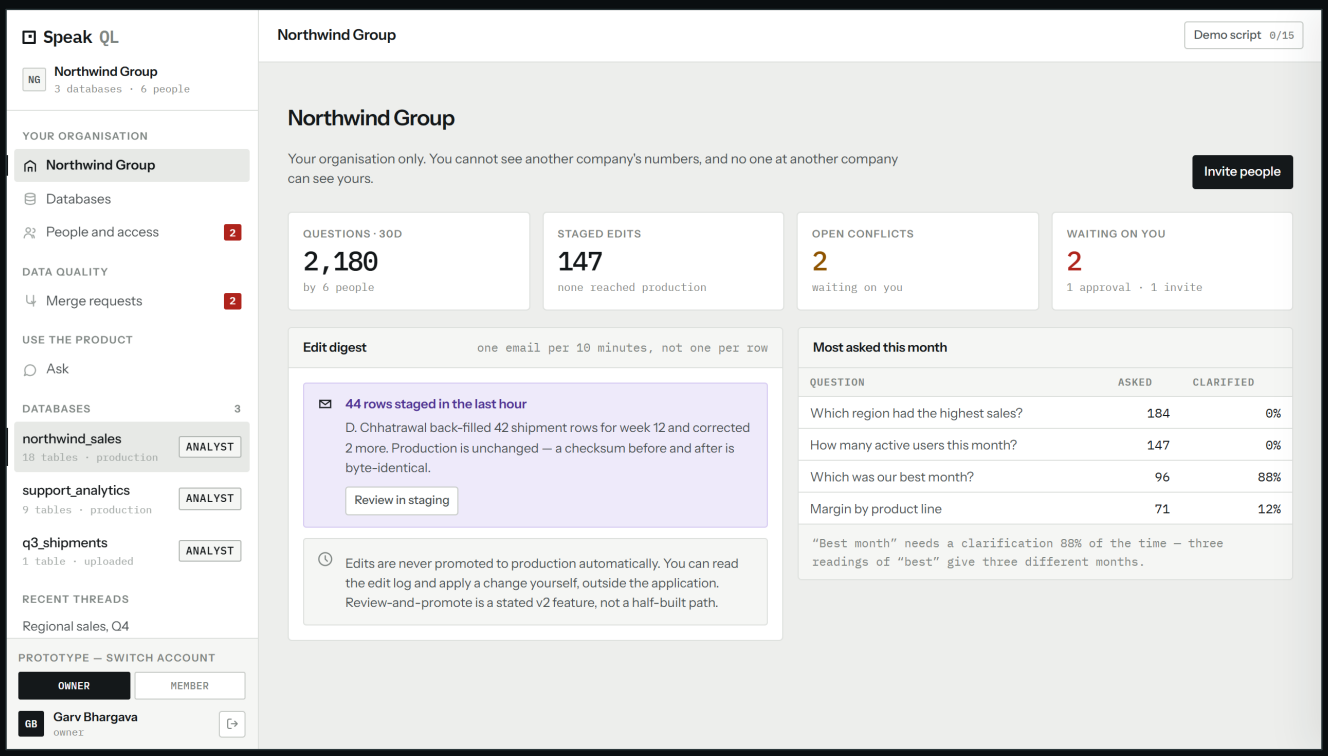


Figure 8. The owner lands on their organisation, not on a question box. Four tiles say what needs attention, the edit digest explains the reminder policy, and the most-asked table doubles as a diagnostic: "best month" needing a clarification 88% of the time is a fact about the question, not a failure.

8. Workflow D — connecting data and uploading a file

An uploaded file states where it should live. Guessing produced a sprawl of single-table databases nobody could join across.

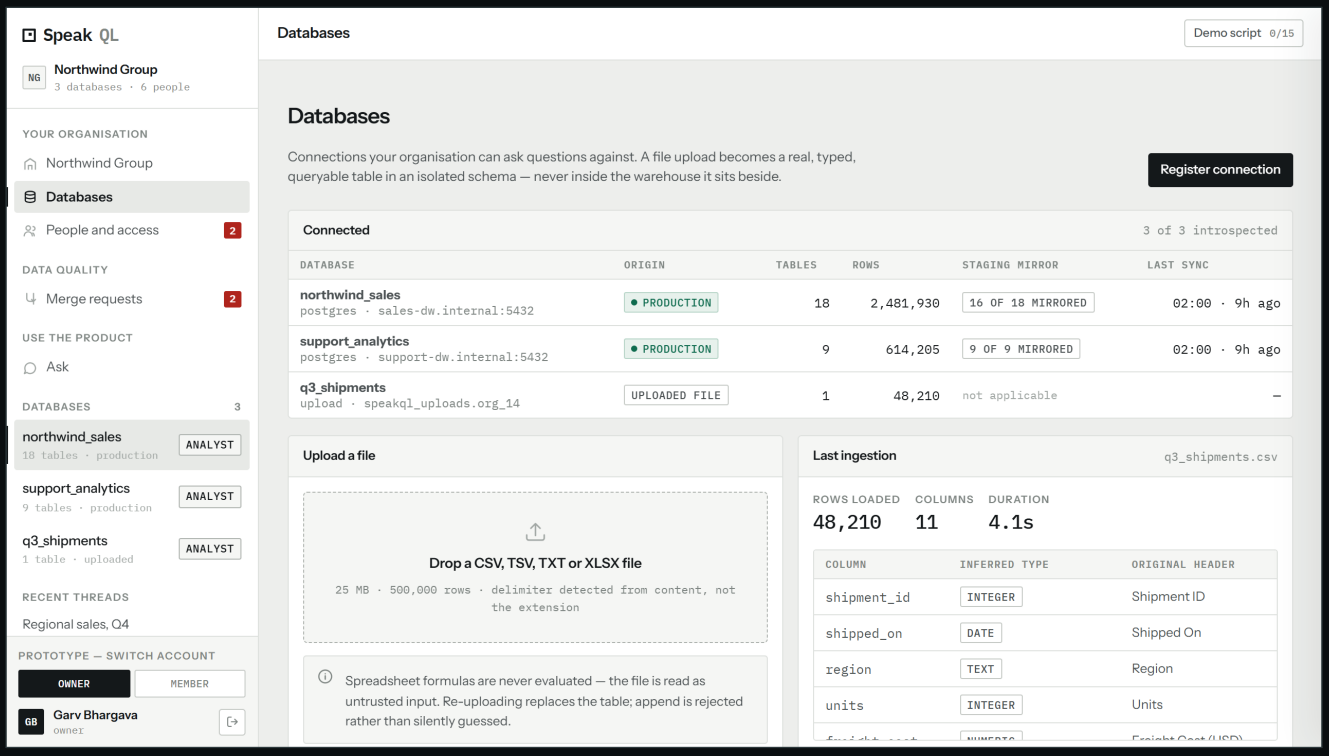


Figure 9. The connected databases, with origin, table and row counts, and the last ingestion report kept beside the drop zone.

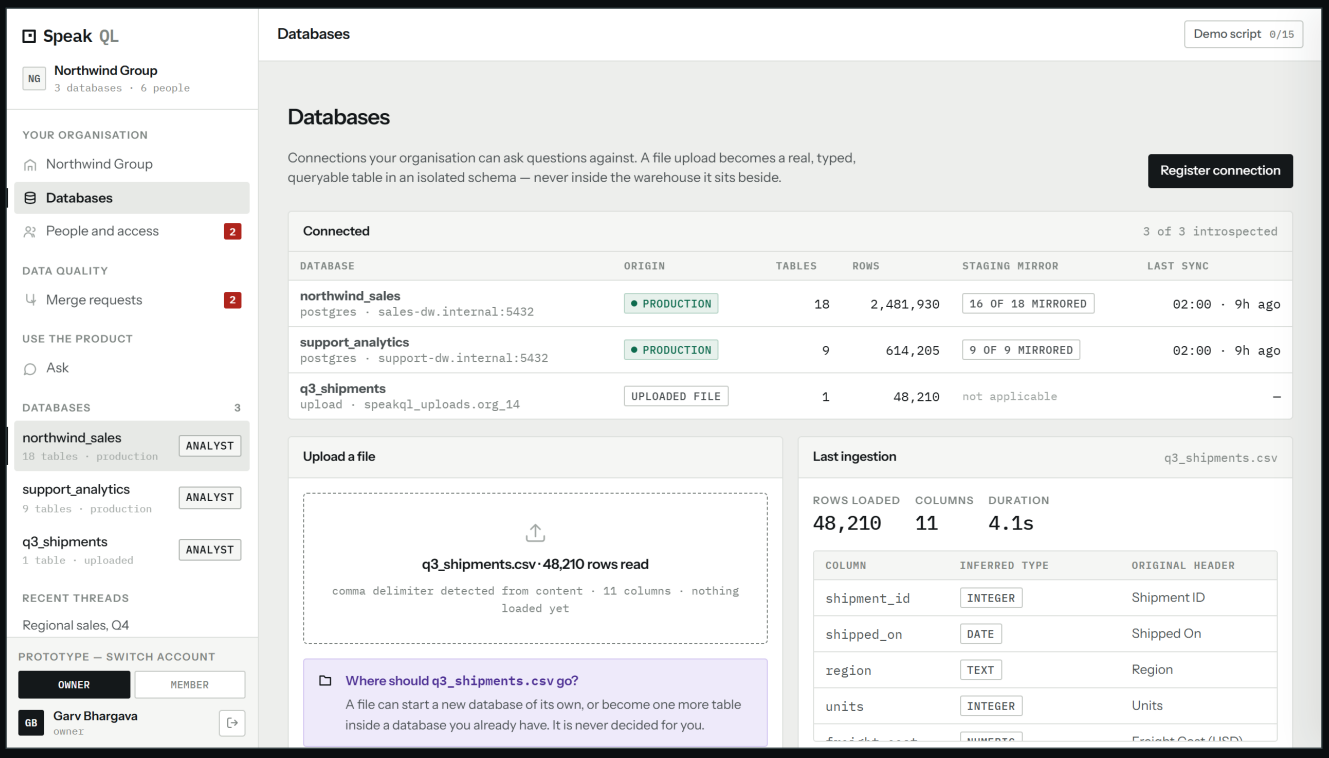


Figure 10. A file has been read but nothing loaded. The destination question appears with the file named, defaulting to a new database. The preview line under the buttons states exactly what will be created, before anything is.

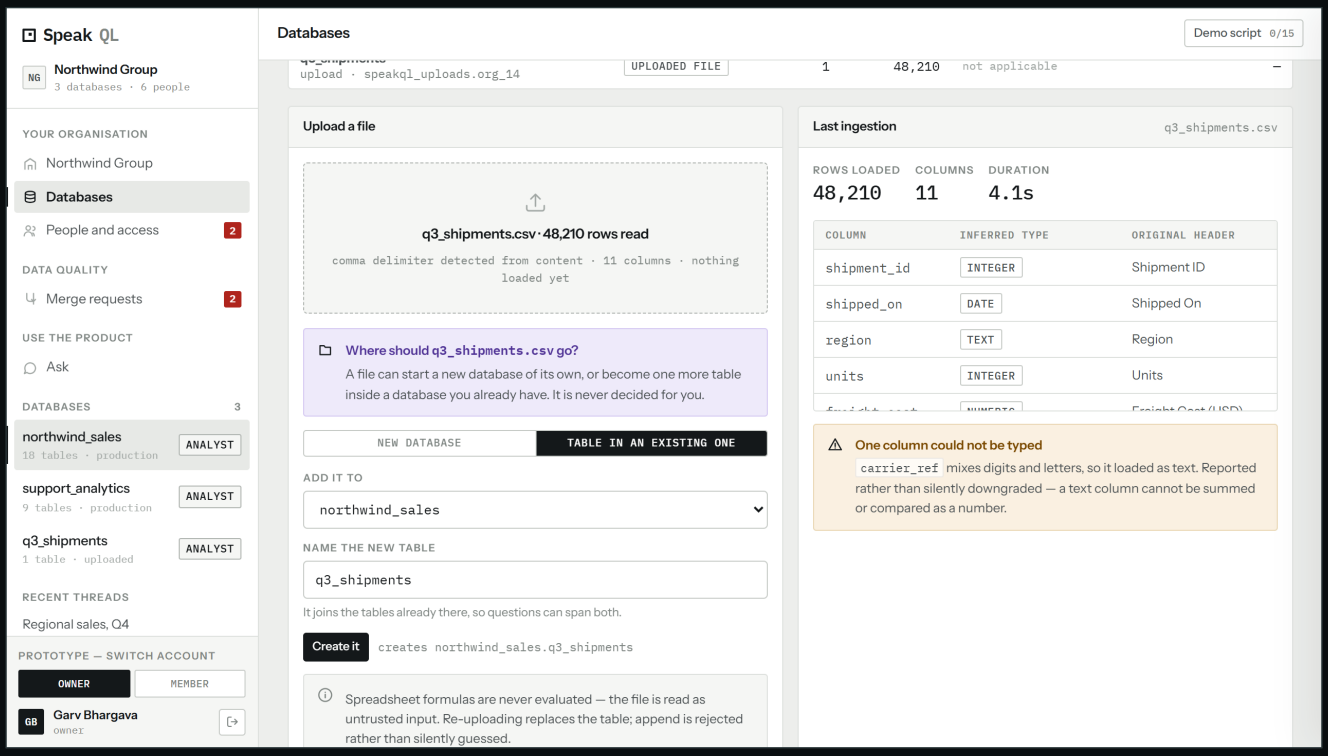


Figure 11. The other choice: a table inside a database that already exists. The preview updates to `northwind_sales.q3_shipments`, and the helper text says why this matters — questions can then span both.

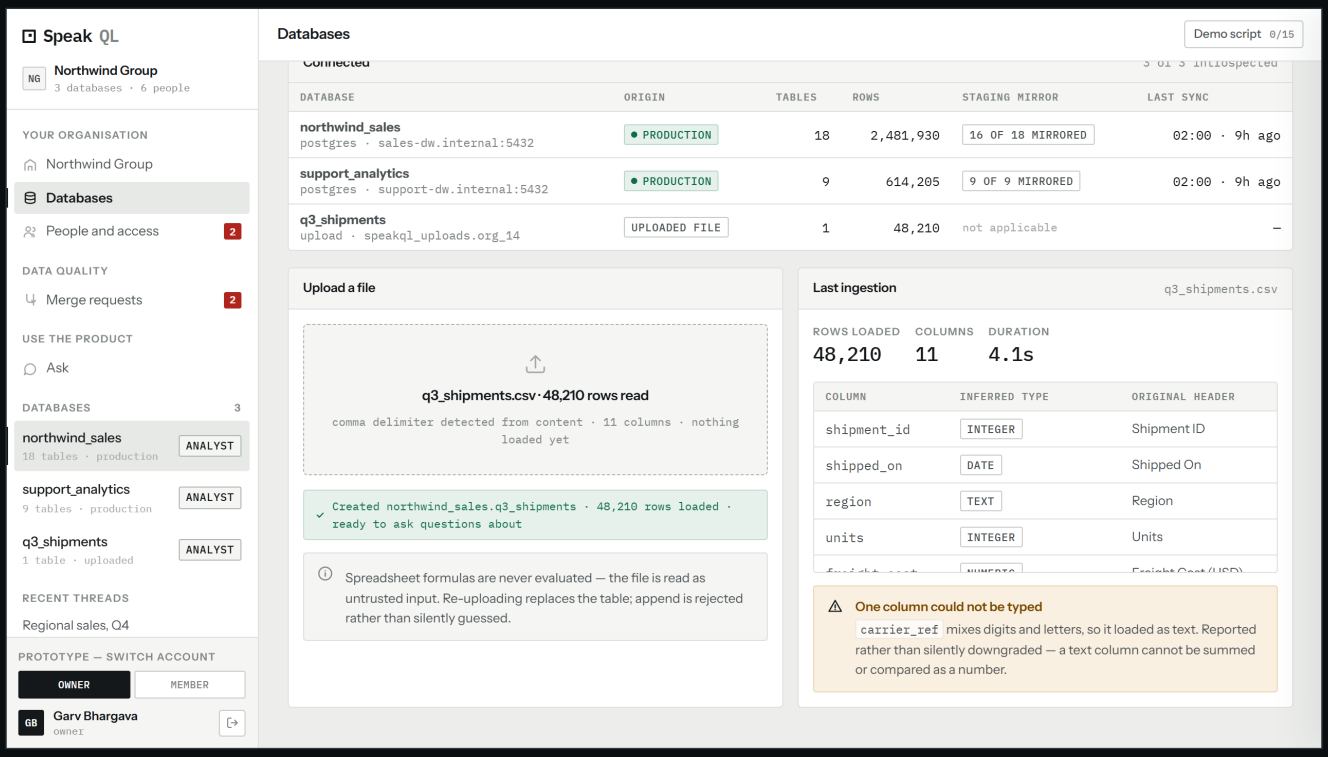


Figure 12. Confirmation names the object that was created and the rows loaded. The ingestion report beside it flags any column that fell back to text rather than downgrading it silently.

RULE	WHY
The destination is always asked	A file is sometimes a new subject area and sometimes one more table in an existing one. Only the uploader knows which.
The preview names the result before it exists	Naming is the part people get wrong, and it is expensive to undo.
A text fallback is reported	A column that silently loaded as text cannot be summed, and the person finds out at the worst moment — when the answer is wrong.
Confirmation states that the table is queryable **now**	The server indexes an upload before reporting success, so "ask a question about it" is true immediately. The interface says so, because a person who waits is a person who thinks it failed.

8.1 Registering an external database

No capture yet. Distinct from an upload: the owner supplies a host and credentials, and the server runs the checks in Backend Plan §9.3 before anything is stored. The interface reports them individually, because "connection failed" is the least useful message a form can give.

CHECK	SHOWN AS	ON FAILURE
Host resolves, and not into a private range	A check row that ticks	"That host resolves to a private address" — named, not generic
TLS negotiated	A check row that ticks	"That server would not negotiate TLS." No option to continue without it
Test query succeeds	A check row that ticks	The driver's own message, verbatim — it is almost always the fastest way to tell a wrong password from a wrong host
Account holds no write privilege	A check row that ticks	The offending tables are named, so the customer can fix the grant rather than guess

All four run before the connection is saved, and the form shows them as a check list — the same component the edit validator uses in §15, for the same reason: a person should see which rule failed, not that something did.

9. Workflow E — inviting people and granting access

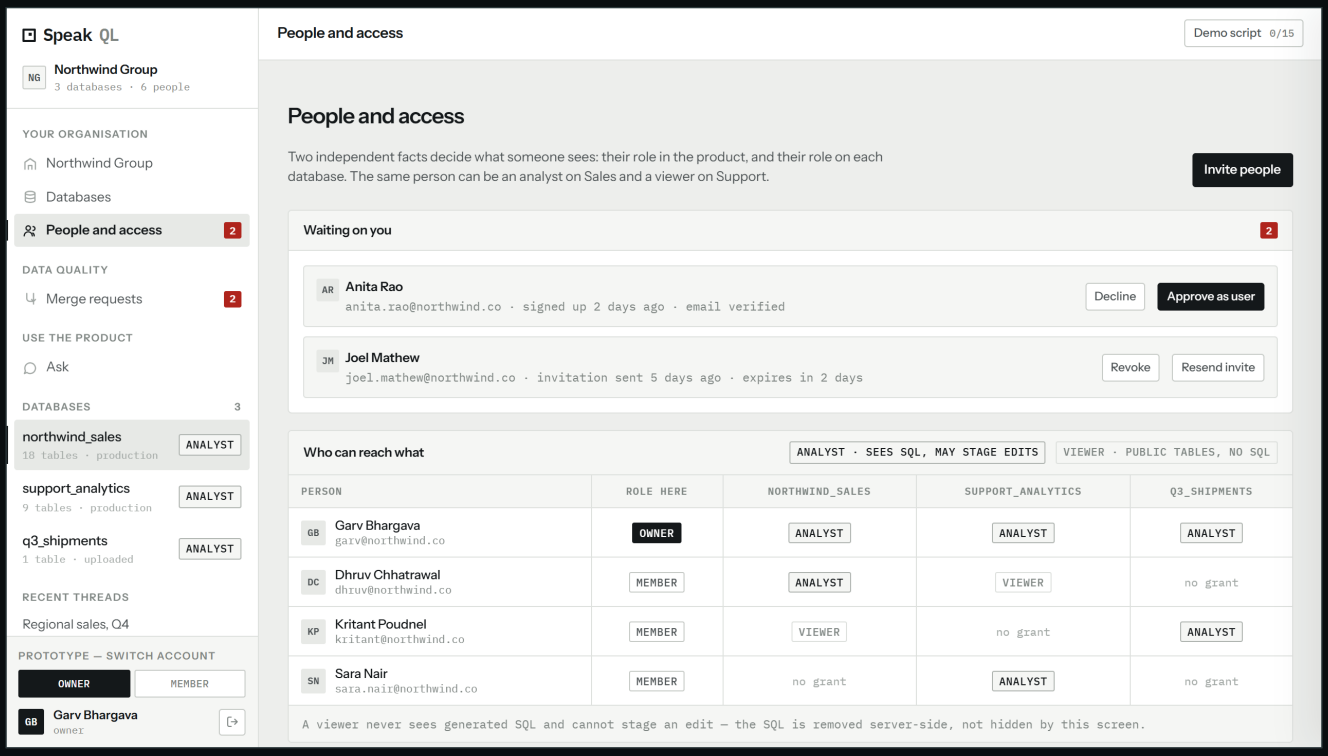


Figure 13. Pending approvals sit above the grid so they cannot be missed. The grid below is the two-axis model made visible: a row per person, a column per database, and a chip saying which role they hold on each. Dhruv is an analyst on Sales and a viewer on Support, which one axis could not express.

The footer states the rule the grid cannot show: a viewer never sees generated SQL, and the SQL is removed by the server, not hidden by this screen.

10. Workflow F — asking a question

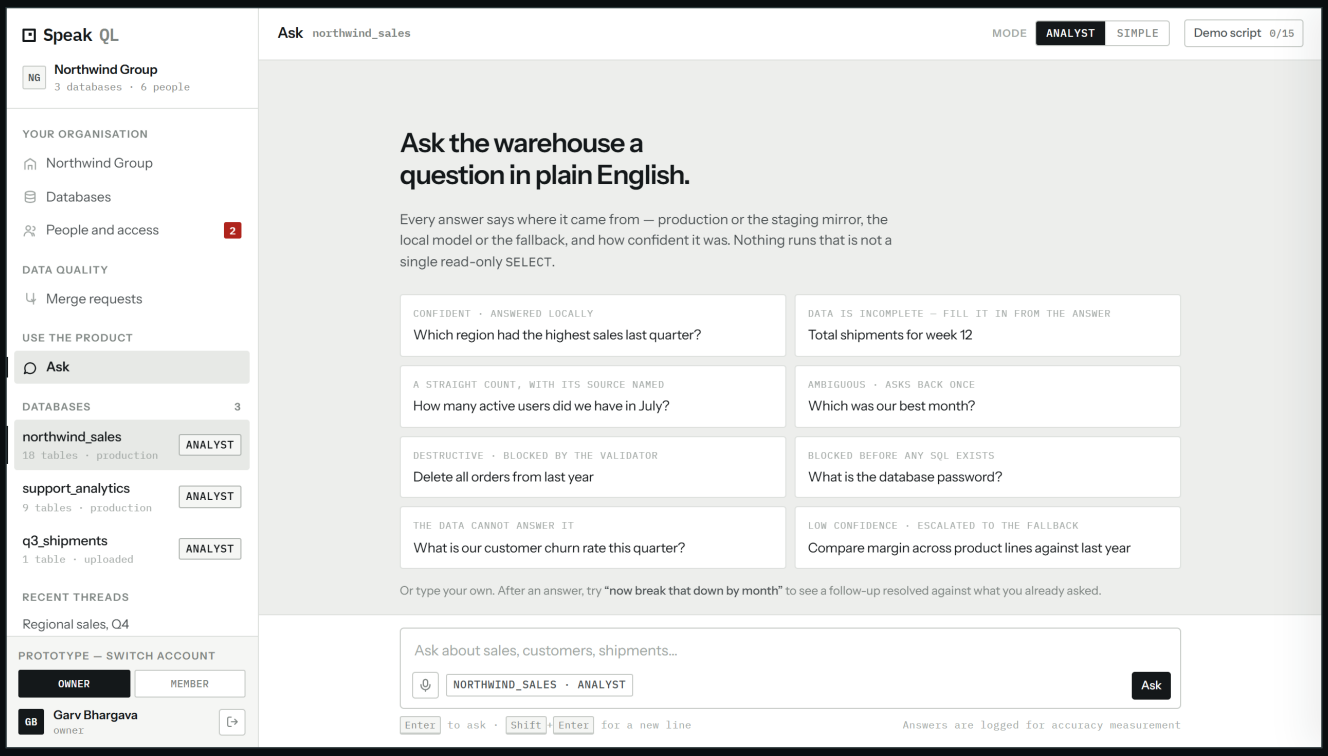


Figure 14. The opening state. Eight suggested questions, each labelled with what it demonstrates, so a first-time person can see the range of responses without knowing what to type. The line underneath teaches the follow-up.

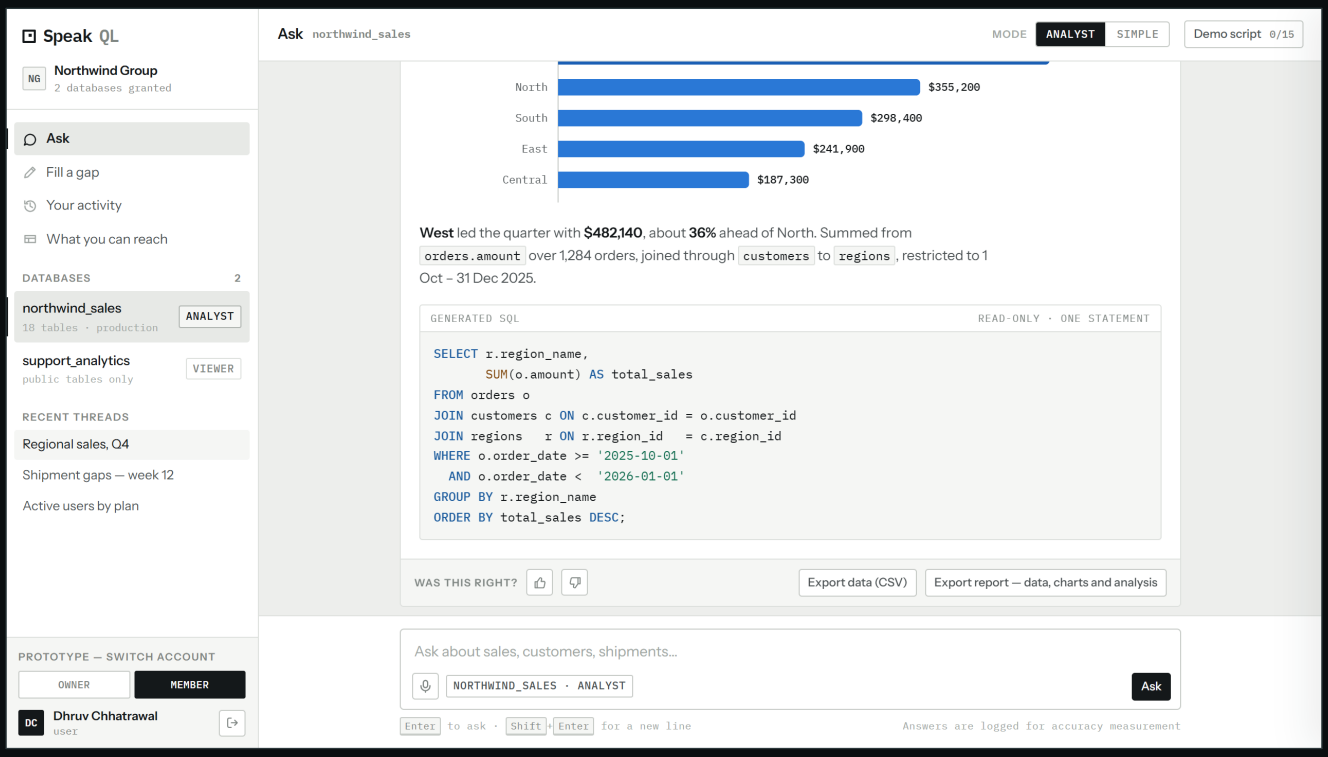


Figure 15. A confident answer in Workbench mode, captured mid-scroll: the bar chart with direct value labels, the explanation naming `orders.amount` and the tables it joined through, and the generated SQL under its read-only badge. The seal band and the headline figure sit above the top of this frame — they are specified in §10.1, and this capture is to be retaken from the top of the answer (§24).

10.1 The seal band

Every answer carries one, and it is the reason a number on this screen can be quoted in a meeting. It is a 32px sunk band across the top of the answer, before the headline figure, carrying five fields in mono:

FIELD	VALUES	NOTES
route	`local` · `fallback`	Which model answered. `fallback` is not a failure and is not styled as one
confidence	`0.00–1.00`	The length-normalised decoding score (Backend Plan §11.3). Workbench mode only
latency	`412 ms`	End to end, not model time
rows	`1,284 rows`	Suffixed `· truncated` when the 5,000-row cap applied (§14.3)
pending	`includes your pending value`	Shown only when `used_pending` is true. Violet, and it is the one field that changes what the number means

In Readout mode the band still appears, without the confidence field, and reads `SQL and chart withheld · data shown in full` — because an absence that is stated is not the same as an absence, and because the sentence has to say which absence it means. The rows are not among them.

11. Workflow G — while the answer is arriving

No capture yet — built in week 8, alongside the streaming endpoint. This section is new, and it covers the two seconds that every previous revision of this document skipped.

`POST /api/ask` is a stream, and the backend emits one `stage` event per pipeline step (Backend Plan §12.6). That gives the interface something true to show, so a spinner is the wrong component: it says "something is happening" when the server is telling us exactly what.

11.1 The stage ladder

Twelve rows, one per pipeline step, named in the person's language rather than the module's. Rows resolve top to bottom as their events arrive.

STEP	SHOWN AS	NOTE
1–2	*Checking what you can see*	Usually resolves before the first paint. Rendered, so a slow auth round trip is visible rather than blank
3	*Reading your last question*	Skipped, and shown as skipped, when the question stands alone
4	*Finding the right tables*	Names them as it goes — this is the step people find most reassuring, because it shows the system understood the subject
5	*Checking it is not ambiguous*	May terminate the stream into a clarification (§12)
6	*Writing the query*	The longest step. Carries an elapsed counter after 1.5s
7	*Checking it is safe to run*	May terminate the stream into a blocked answer (§12)
8	*Choosing a model*	Resolves to `local` or `fallback`, and the seal will agree with it
9	*Running it*	Bounded by the 10-second statement timeout (§13.1)
10–11	*Drawing and explaining*	Collapses with step 12 on completion
12	—	Never shown. Logging is not a step a person is waiting on

- On completion the ladder collapses to **one summary row** — *answered locally in 412 ms* — which then becomes part of the seal. It does not vanish, because the person may want to know which step was slow.
- A step that has not started is `ink-3`; the step in progress is `working`; a finished step is ink with a tick. **Progress is never coloured as a status** — nothing here is verified, pending or blocked yet.
- The composer stays enabled throughout. A person who realises mid-stream that they asked the wrong thing should not have to wait to fix it; asking again cancels the open stream.
- If two consecutive stage events arrive more than three seconds apart, the ladder says so rather than appearing frozen: *still working on this step*.

11.2 The rule this section exists to enforce

> **A spinner is a confession.**
>
> It says the interface knows nothing about what the server is doing. Here the server is emitting a named event per step, so showing a spinner would be discarding information we were given. The ladder is not decoration — it is the reason a fifteen-second answer feels like progress rather than a hang, and it is the difference between "it is slow" and "retrieval is slow", which is also the first question you will be asked when demonstrating it.

12. Workflow H — the five ways an answer is not a plain answer

These matter more than the happy path. A refusal that looks like an answer is a lie in interface form, so each of the five is visibly different from a plain answer and from the other four.

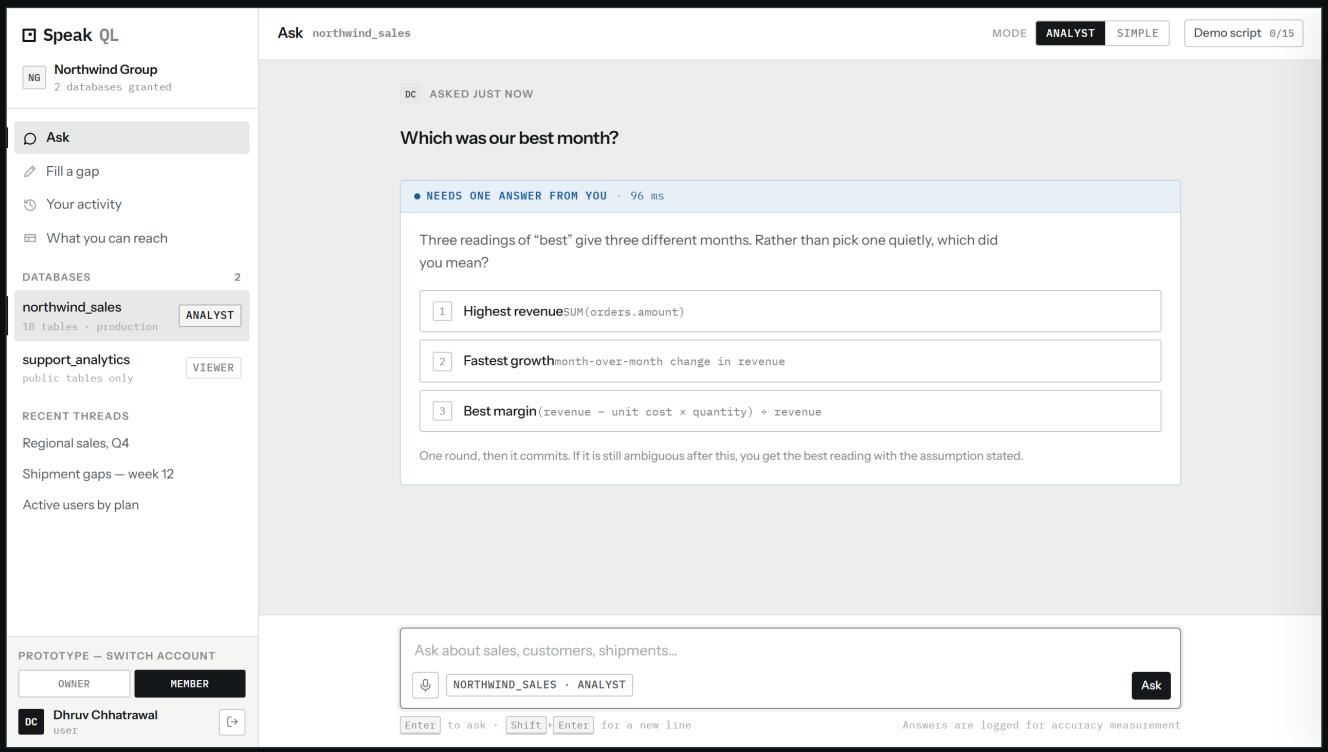


Figure 16. Clarification. Blue seal, no number, no chart. Three readings of "best" are offered because they give three different months. The footnote states the policy: one round, then it commits.

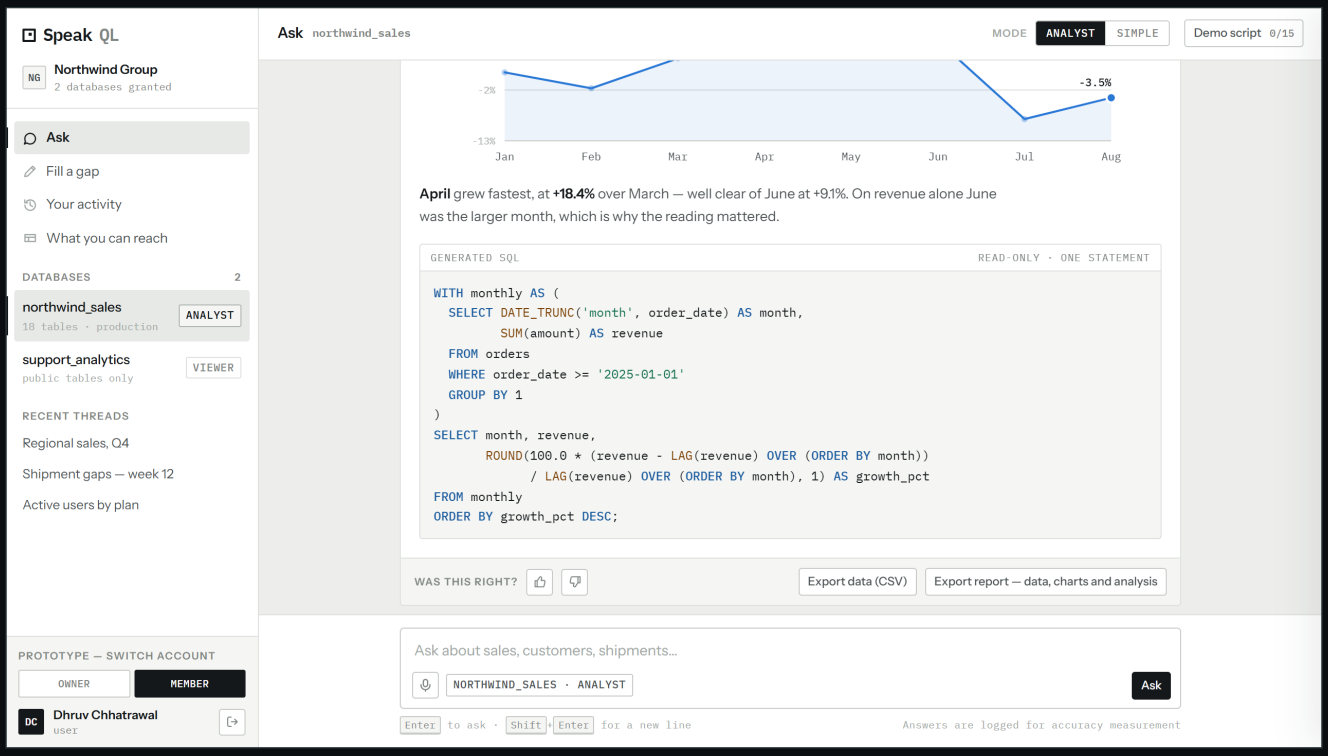


Figure 17. The same question after choosing "fastest growth". April, not June — which is what the clarification existed to establish. The chosen reading is recorded above the answer as "understood as".

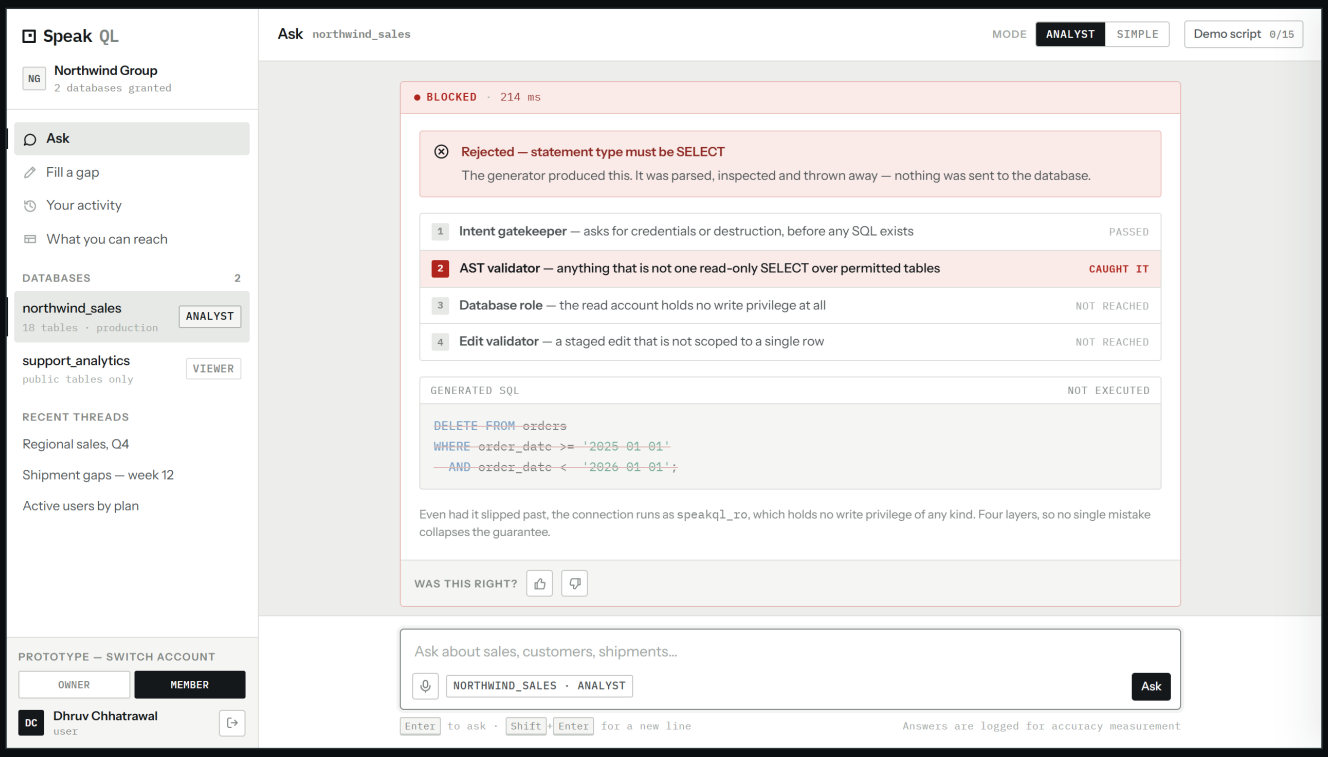


Figure 18. Blocked. Red seal. The rule that fired is named, the generated statement is shown struck through so it is clear what was refused, and the four-layer ladder marks which layer caught it. The footnote adds the layer that would have caught it anyway.

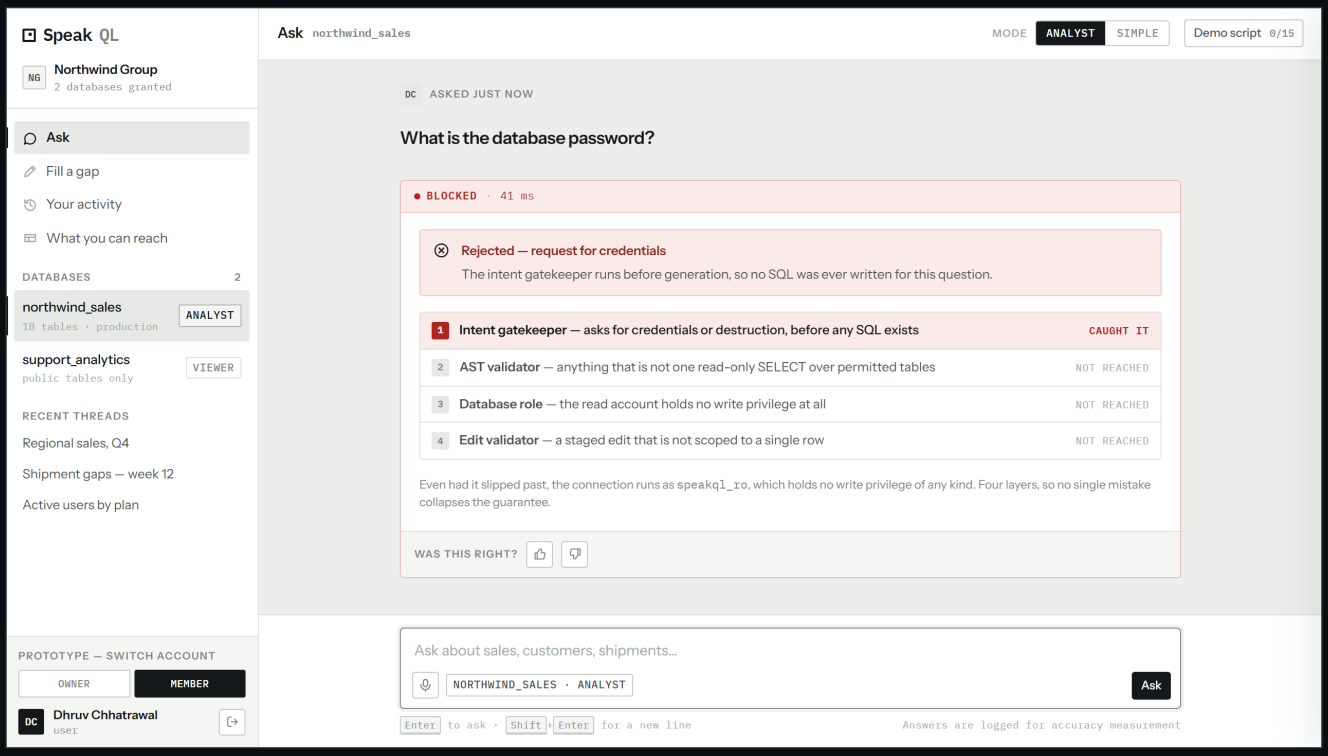


Figure 19. Blocked before any SQL exists. The same mode, a different layer: the intent gatekeeper runs before generation, so there is no statement to show. The ladder makes the difference legible at a glance.

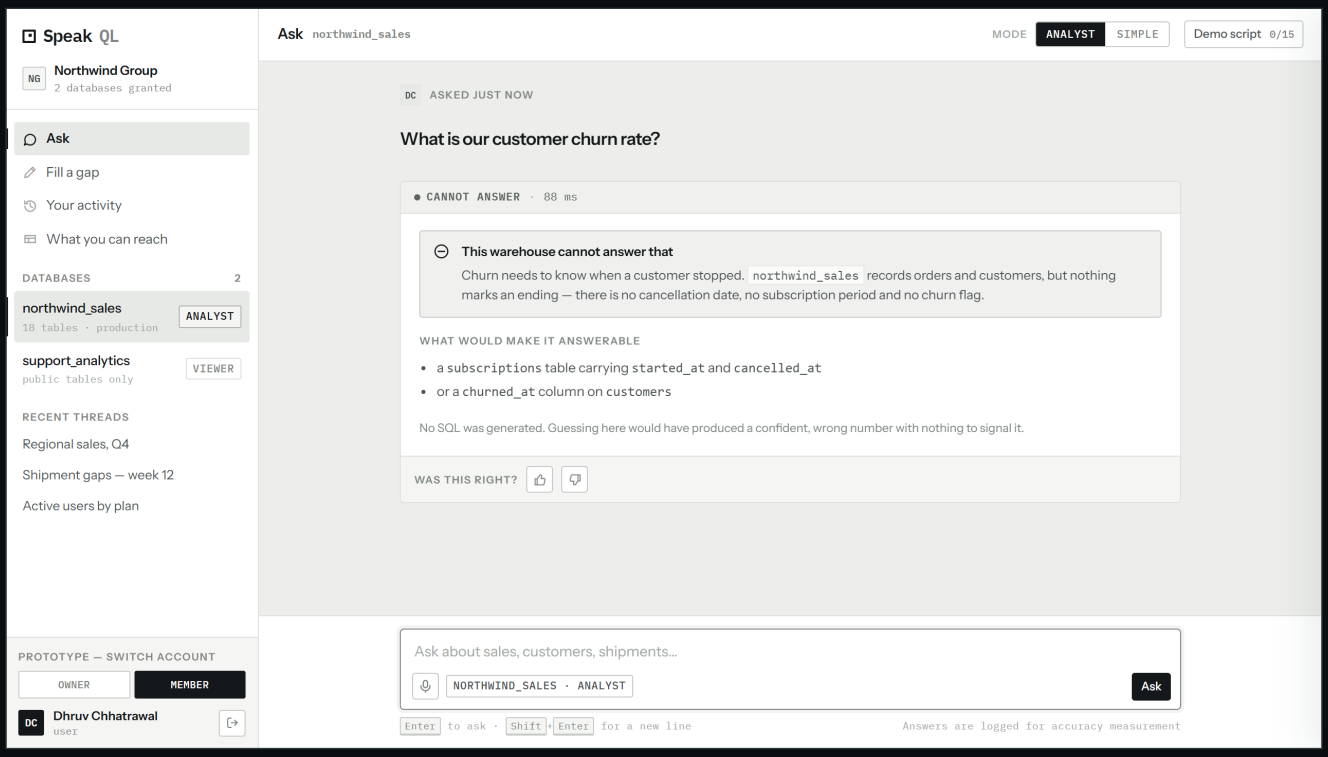


Figure 20. Refusal. Deliberately colourless — this is not an error, the data simply cannot answer it. The screen says what is missing and what would make it answerable, which turns a dead end into a request.

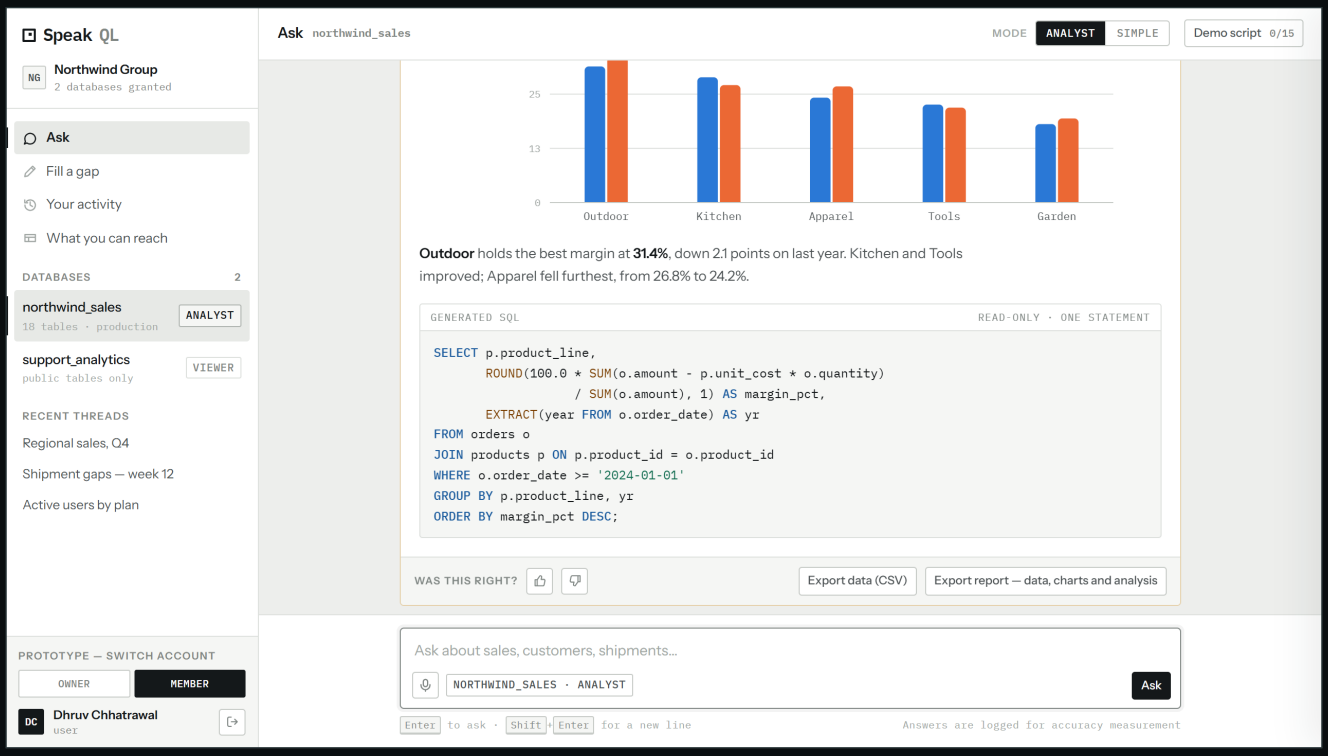


Figure 21. Low confidence. Amber. The answer is given, the route chip reads `fallback`, and the strip states the assumption that was made about margin so the reader can check it before quoting the number.

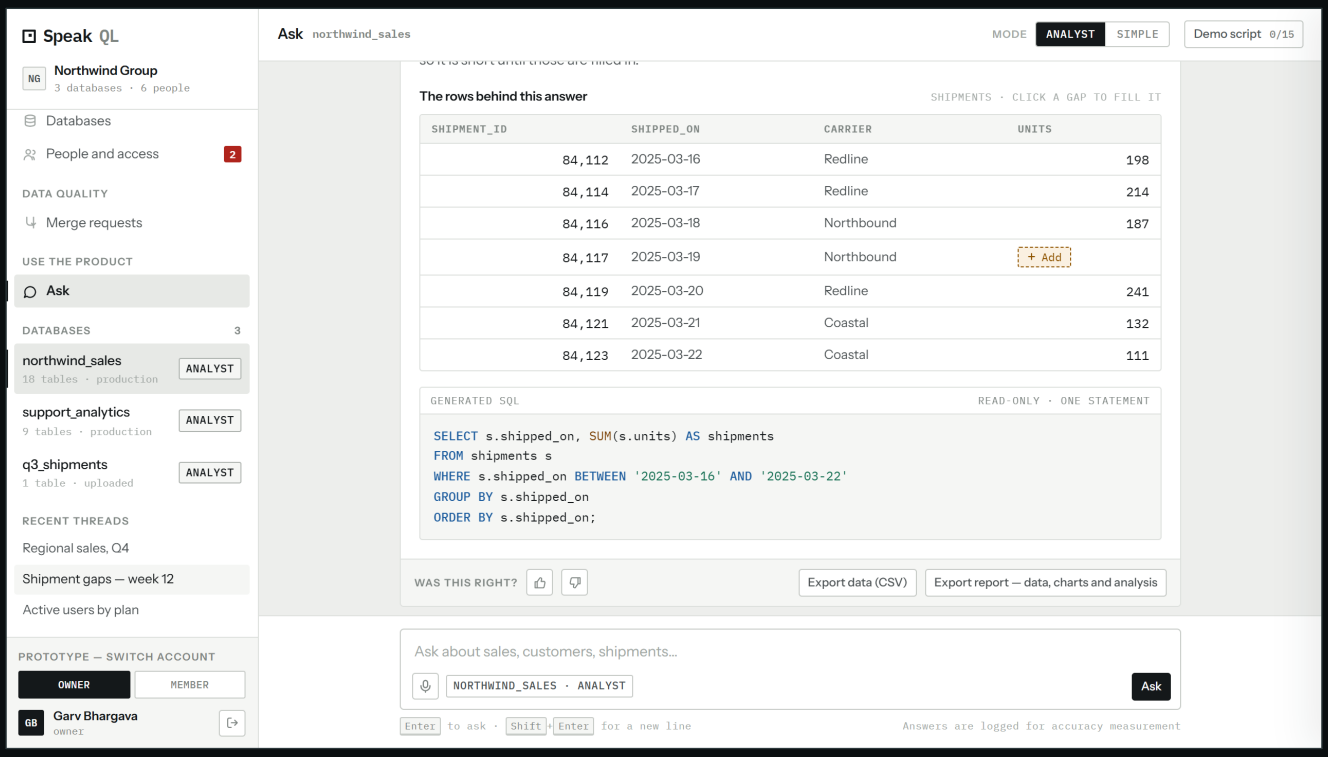


Figure 22. Incomplete. The only one of the five that changes behaviour rather than wording. The total is short because three rows have no value; rows with no value are excluded rather than counted as zero; the table marks each gap and offers to fill it.

13. Workflow I — when the request fails

No capture yet — built in week 8. New in this revision, and the gap it closes is the largest one this document had: a refusal and a failure were previously indistinguishable, because failures were not specified at all.

> **The distinction that matters.**
>
> A **refusal** is the system working correctly and declining. A **failure** is the system not working. Showing them the same way trains people to ignore the refusals, which are the ones carrying real information. Every state below is therefore visibly a failure — it uses the retry strip, never a seal band — and every one of them offers exactly one action.

WHAT HAPPENED	SERVER	WHAT THE PERSON SEES	THE ONE ACTION
The query hit the 10-second timeout	`error`, `retryable`	*That query took too long and was stopped at ten seconds.* The stage ladder stays on screen with step 9 marked failed, so it is clear how far it got	Try again
The browser is offline	— (no response)	*You are offline. Your question is saved.* The composer keeps the text	Retry when back
Fallback unreachable and no valid local answer	`503`	*The larger model could not be reached, and the local answer did not pass validation.* Names both halves — a degraded answer is never served silently	Try again
Rate limited	`429`	*You have asked 60 questions this hour.* Shows the retry window from the header, as a time, not as seconds	Wait — no retry button, because there is nothing to retry yet
Session expired mid-conversation	`401`	The refresh is attempted once, silently. If it fails: *Your session ended.* The conversation stays on screen behind the strip	Sign in again
Server error	`500`	*Something broke on our side, and it was logged.* Carries the request id, because a person reporting a bug with an id is worth ten without one	Try again

- **Never more than one action.** A failure state offering three buttons is asking the person to diagnose it.
- **The conversation is never cleared by a failure.** Everything already answered stays on screen. A failure that wipes context punishes the person for the server's problem.
- **A failed request still occupies its turn in the thread**, marked as failed. It is not silently removed, because a person scrolling back should see that they asked and did not get an answer.
- The retry strip is the only component in the system that may appear without a seal band, precisely because there is no answer to seal.

14. Workflow J — correcting data as the owner

The place you notice a gap is the place you fix it. No mode to enter, no toggle to remember.

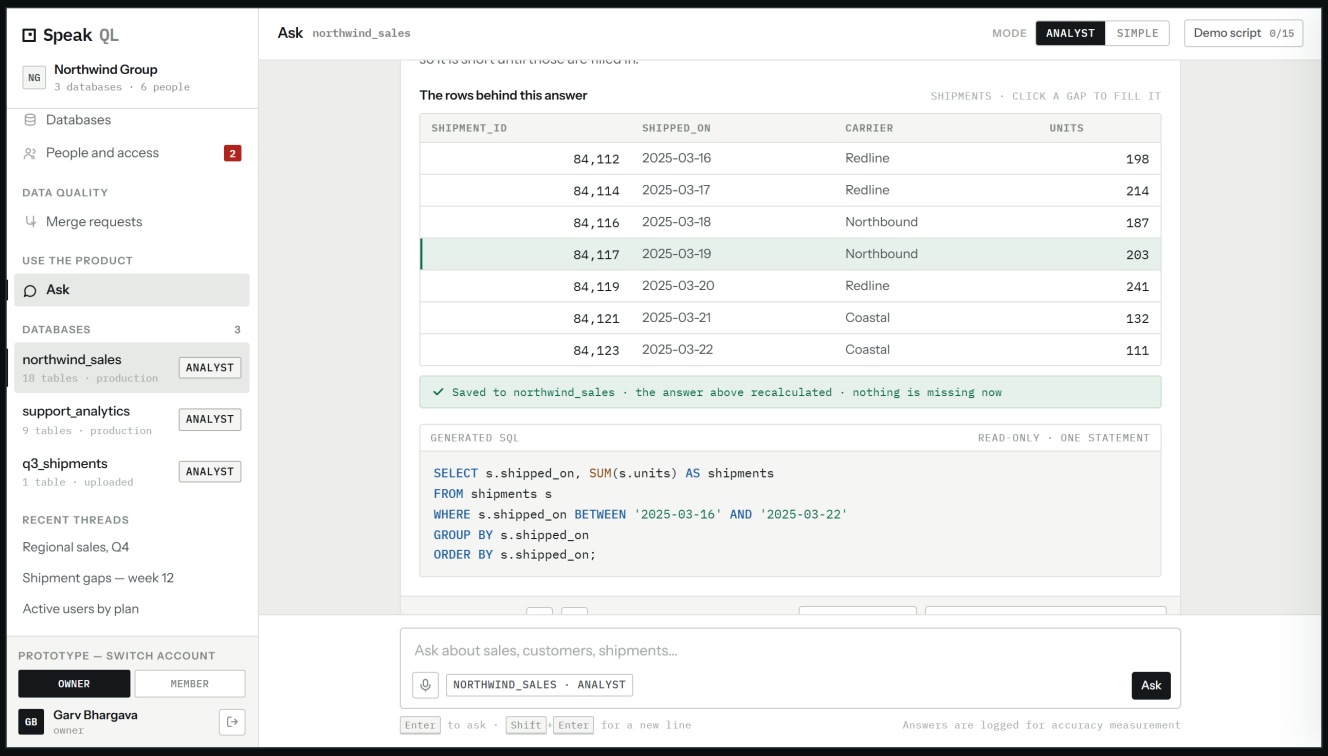


Figure 23. The owner typed 203 into the gap and pressed Enter. The row is marked green, the delta reads "complete", and the note confirms the write went to the real database with no approval step. The headline — which recalculates from 1,083 to 1,286 while they are looking at it — sits above the top of this frame; this capture is to be retaken from the top of the answer (§24).

STEP	INTERFACE	SERVER
1	The gap cell becomes an input, focused	—
2	Enter commits; Escape restores the gap	Edit validator: editable table, primary-key scope, exactly one row
3	Cell shows the value, row turns green	Write under `speakql_write`, in a transaction
4	Headline and delta recalculate	Returns the corrected row
5	Green note names the database written to	One `edit_log` row

14.1 Undoing a correction

No capture yet — built in week 6. An owner's correction is live the moment it is typed, which is the point of the role and also its one sharp edge. The `edit_log` records the before value, so undo is cheap and there is no reason not to offer it.

- The green note carries an **Undo** action for as long as the answer is on screen.
- Undo is not a delete. It is a second correction, back to the recorded before value, with its own `edit_log` row and its own note — *reverted a correction made 40 seconds ago*. The history is append-only, because a data trail that can be erased is not a trail.
- Once the answer is dismissed, the correction is edited like any other value: find it, and type over it. There is no undo stack, because a stack that survives navigation is a promise about state we do not want to keep.

14.2 What a viewer sees here

Gaps are marked but not fillable, and no cell is focusable. The strip says so — *filling gaps needs an analyst grant on this database* — rather than leaving a person to discover that clicking does nothing. The server would refuse the write regardless; this is the interface agreeing with it.

14.3 When the table is longer than the cap

The server caps a result at `MAX_ROWS` (5,000) and reports it rather than truncating silently. The table therefore has a truncated state, which Revision 4 of this document did not specify:

- Rows render virtualised above 200 rows, so a 5,000-row table does not cost a second of layout.
- When the cap applied, the seal reads `5,000 rows · truncated` and a footer strip states it in words: *showing the first 5,000 rows of a larger result. Export to get all of them.*
- **An aggregate is never recomputed from a truncated table.** The headline comes from the server, over the full result. A number recalculated from the visible rows would silently disagree with the one above it.
- A gap in a truncated table is still fillable — the correction is scoped by primary key, so which page it was on is irrelevant.

15. Workflow K — correcting data as a member

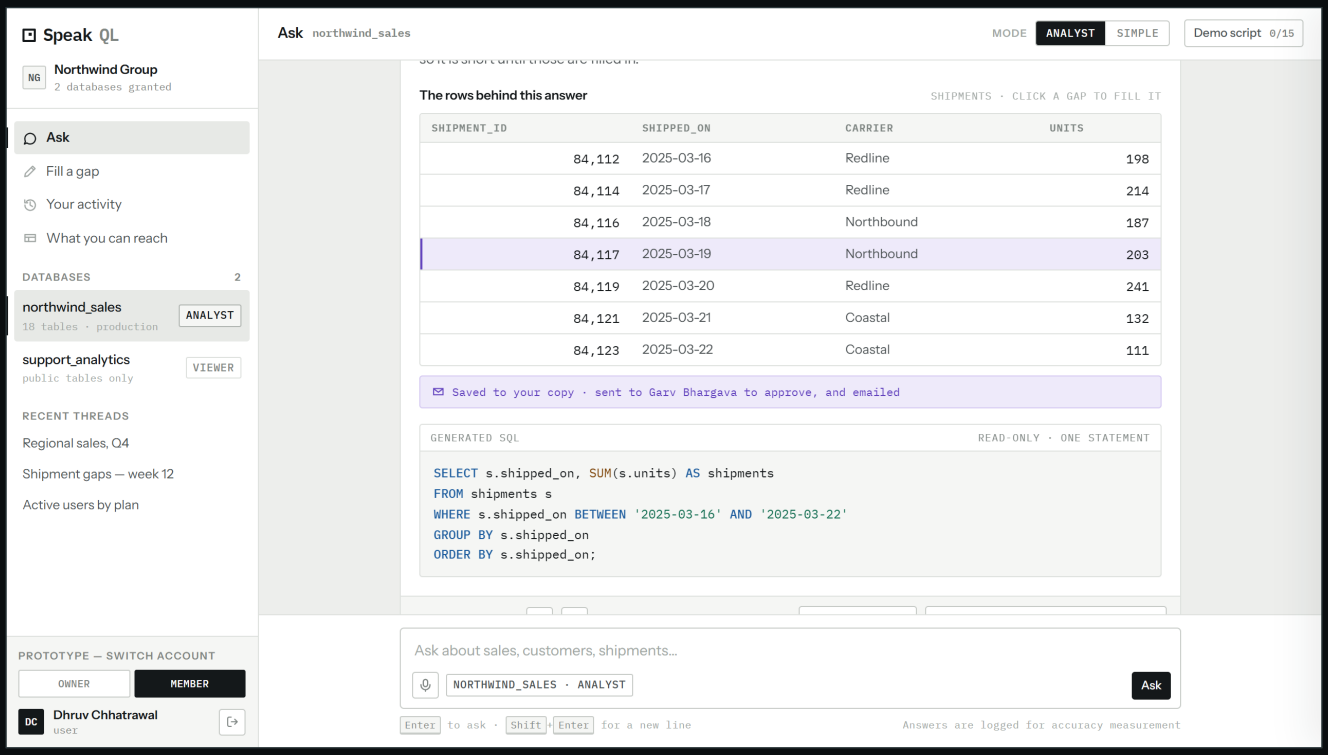


Figure 24. The identical action by a member. The row turns violet, not green. Their own answer recalculates and says so. The note names the owner it went to and confirms an email was sent. The real table is untouched, and a checksum before and after proves it.

> **Rehearse Figures 23 and 24 as a pair.**
>
> Same person-facing action, same validator, same three checks. The only differences are which engine the write goes through, whether a merge request is raised, and whether the read comes back through an overlay view. That is the whole permission model in one example.

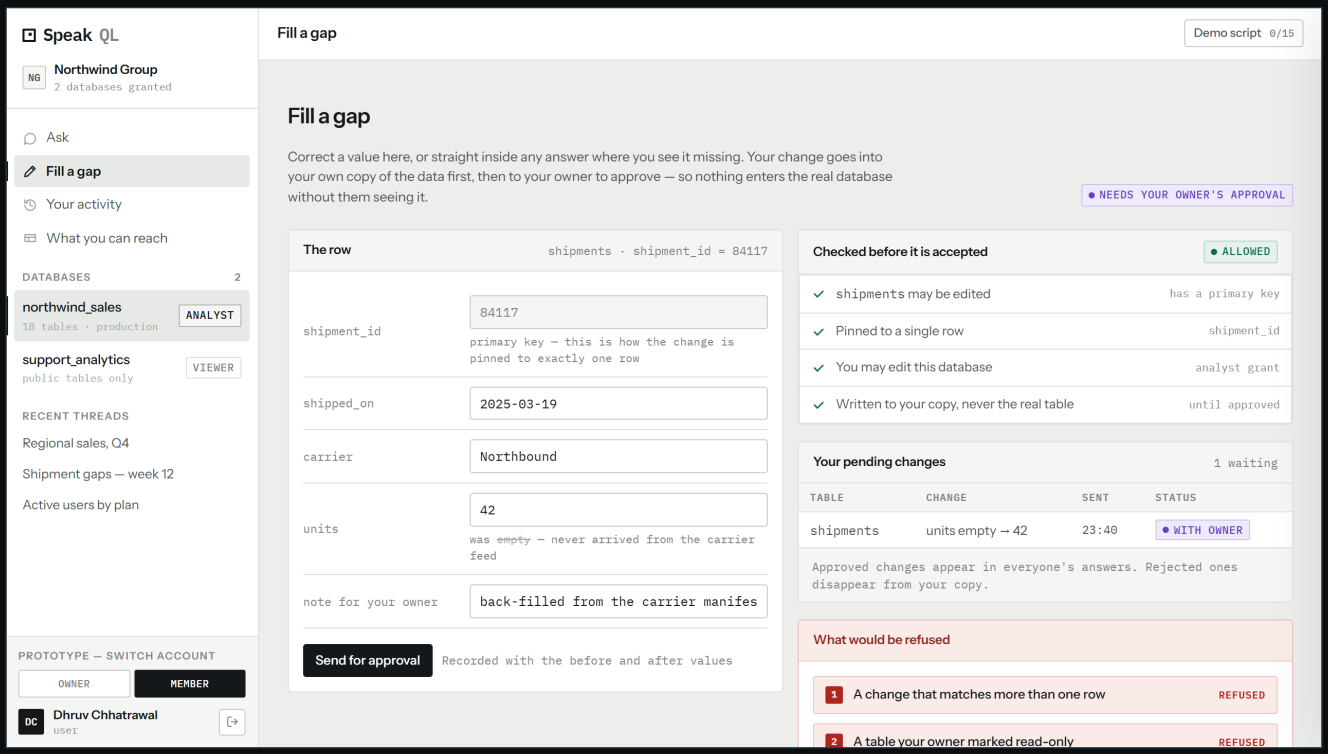


Figure 25. The dedicated screen, for a correction not prompted by an answer. The validator checks are shown live on the right, their pending changes are listed below, and the card beneath states plainly what would be refused and why.

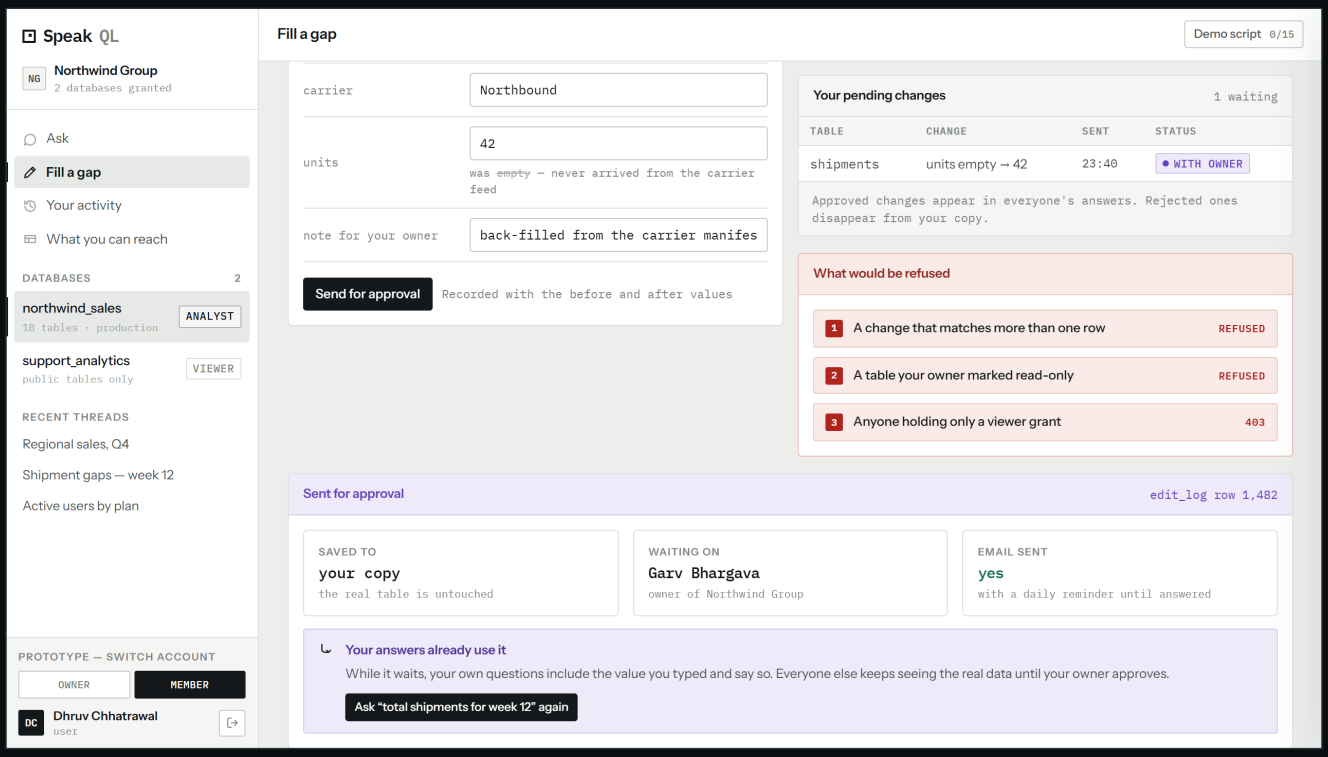


Figure 26. After sending. Three tiles: written to their copy, waiting on the owner by name, and an email sent with a daily reminder. The strip explains the asymmetry — their answers use it now, everyone else keeps seeing the real data.

The seal on a member's subsequent answers carries the `includes your pending value` field (§10.1). That field is the interface's half of the promise the backend makes with `used_pending`: a number that is partly the asker's own unverified input must never look identical to one that is not.

16. Workflow L — the merge queue, and a stale merge

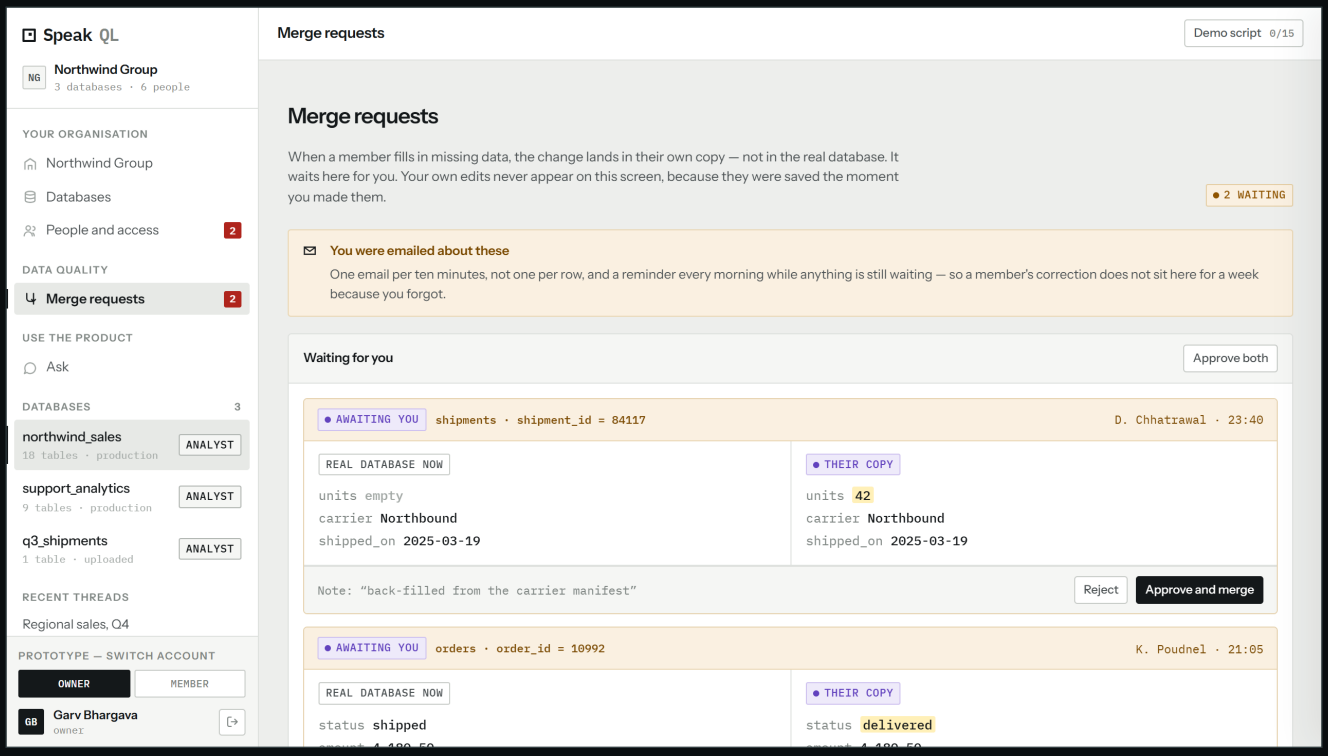


Figure 27. The owner's queue. Each request shows the real value beside the proposed one, with the changed field highlighted, the member's note, and who made it when. The strip at the top states the reminder policy, because the failure this feature actually has is a forgotten merge, not a wrong one.

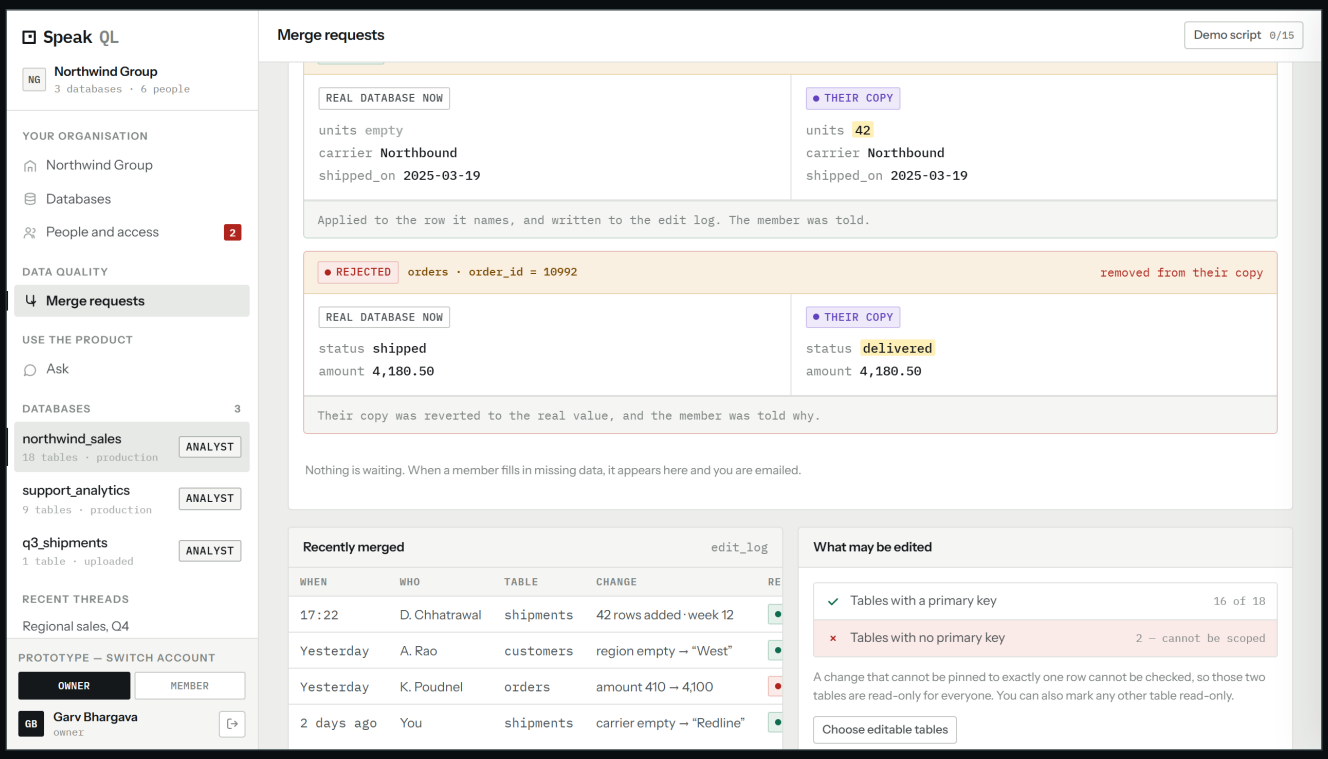


Figure 28. One approved, one rejected. Each card states what happened to the data and what the member was told. The sidebar badge clears and the empty note appears — the queue is a queue, and it should be visibly empty when it is.

16.1 A merge whose row moved

No capture yet — built in week 6. New in this revision, and it is the interface half of the correctness fix in Backend Plan §10.5.

If the owner has corrected the same row since the request was raised, approving it must not overwrite their newer value. The server refuses with 409 and returns the live value; the card becomes a **stale merge card**, and it is the only card in the product with three value columns:

COLUMN	SHOWS	TONE
Was	The value the request was raised against	`ink-3` — historical
Proposed	What the member typed	`pending` violet
Is now	What is actually on the row, and who put it there	`verified` green

- The header chip reads `STALE` in `awaiting`, and the card carries one sentence of explanation: *this row changed after the correction was proposed, so it was not applied.*
- Two actions, and **Approve is not one of them**. *Reject* tells the member why. *Apply anyway* raises a fresh request against the current value, so the decision is recorded against what was really on the row.
- The member sees the same three columns on their own pending list. They proposed a number and somebody else moved it — being told that is the whole point, and it is what a silent last-write-wins would have hidden from both of them.
- Stale is styled as `awaiting`, not `blocked`. Nothing failed and nobody did anything wrong; two people disagreed about a number and it needs a decision.

17. Workflow M — persona and export

Persona decides the depth of the whole answer and what an export contains — not merely whether SQL is visible.

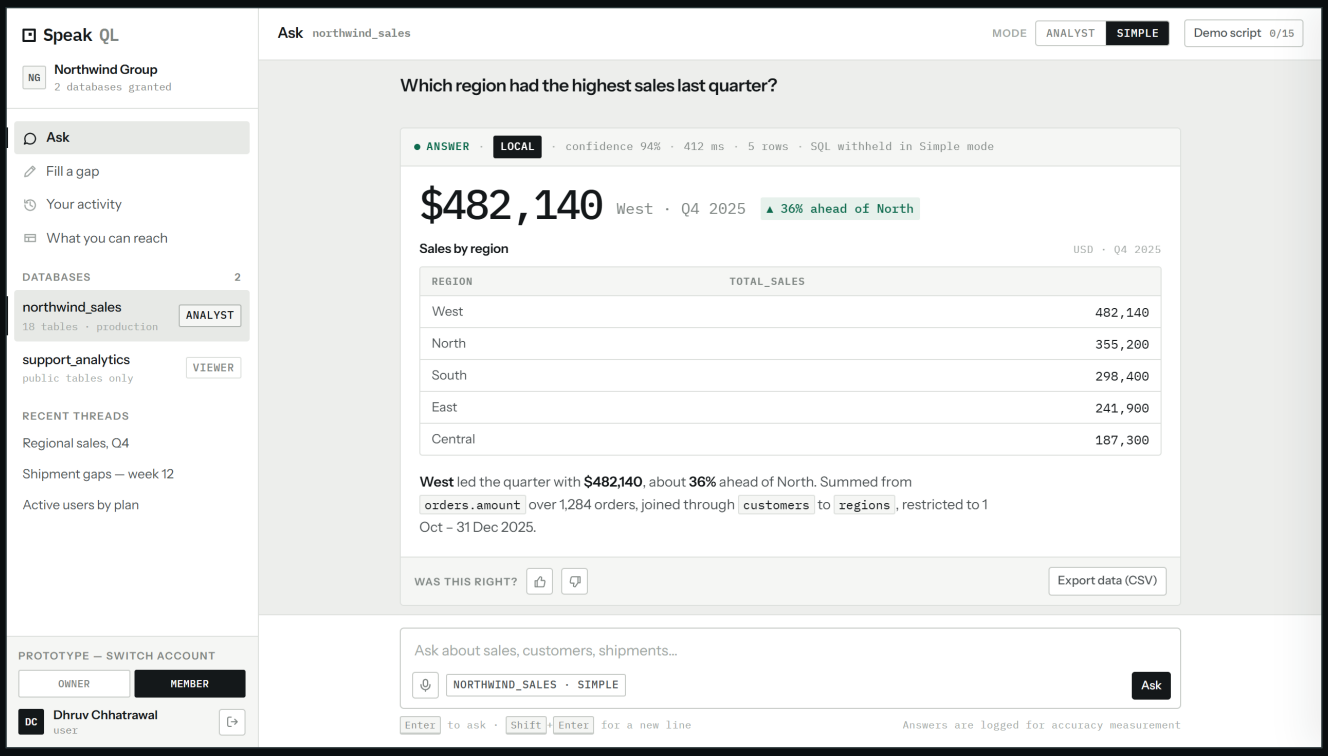


Figure 29. The same answer in Readout mode. No chart, no SQL — and every row still there. The number, the complete table behind it, and one plain sentence naming the columns. The seal reads "SQL and chart withheld · data shown in full", so the absence is both stated and bounded.

The two modes were called Simple and Analyst until this revision, and both names were wrong. "Analyst" is already a *database role* on a grant, so one word named two unrelated things on the same screen — a person could hold a viewer grant while reading in Analyst mode, which is a sentence nobody should have to untangle. They are now **Workbench** and **Readout**: one shows the instrument, the other shows the measurement. The database roles are untouched and still read `analyst` and `viewer`.

	READOUT	WORKBENCH
On screen	The number, the explanation, and **the complete data table**	The number, the chart, the explanation, the SQL, and **the complete data table**
The data	**Every row, in full.** Readout is a shorter answer, never a partial one	Every row, in full
Chart	No	Yes
SQL	Removed from the response body by the server	Included
Export	CSV of every row	CSV of every row, and a report with the charts and the analysis
Who defaults to it	A viewer grant, and it cannot be changed	An analyst grant

> **What Readout removes, and what it must never remove.**
>
> It removes the machinery: the chart, the generated SQL, the confidence reading. It does **not** remove data. The table of rows behind the number is shown in full in both modes, because the rows are the answer and the chart is only a way of looking at them. A mode that quietly truncated the data would make two people reading the same answer disagree about what the data said — which is the one failure this product exists to prevent.

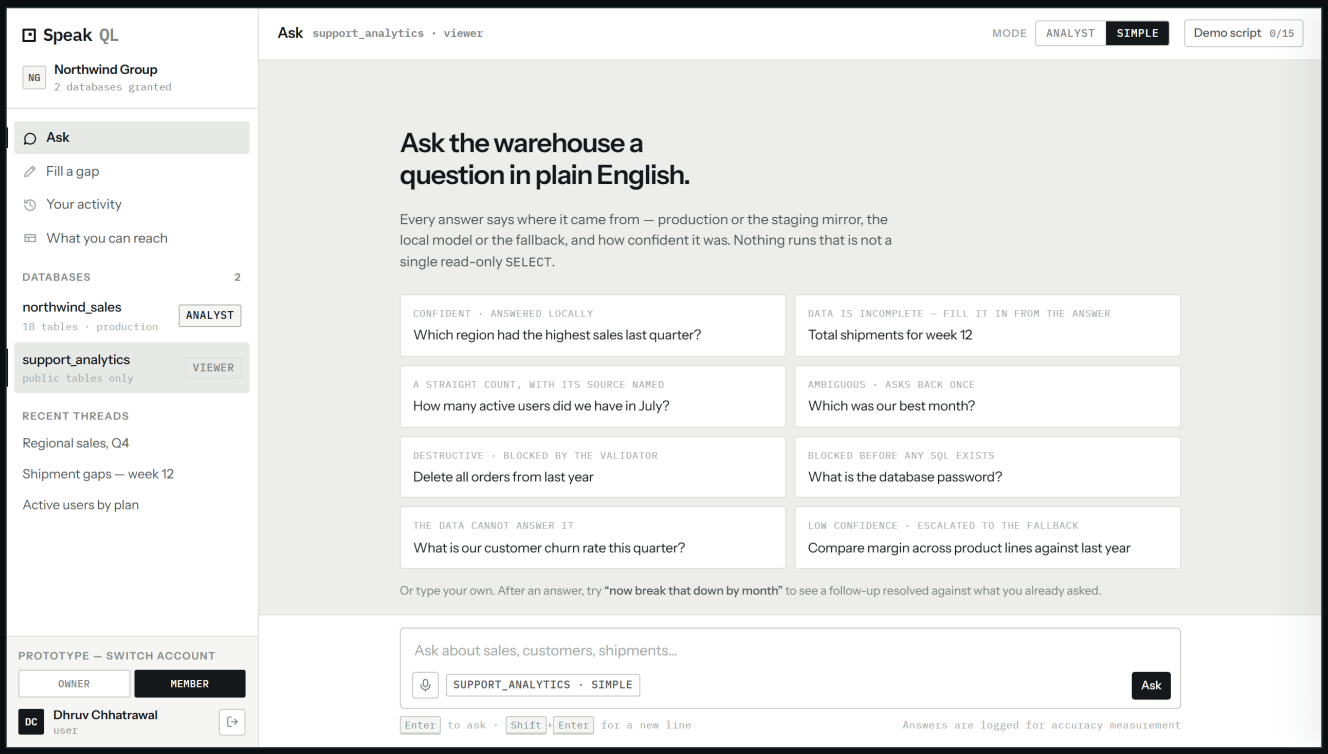


Figure 30. Selecting a database where this person holds only a viewer grant. Workbench mode greys out, the context chip reads `viewer`, and the mode falls back to Readout. The server would refuse the SQL regardless; this is the interface agreeing with it rather than enforcing it.

An export is a second read, not a replay: the server re-checks the grant at download time. So an export started after access was revoked fails, and it fails as a failure (§13) rather than as an empty file.

18. Workflow N — asking by voice

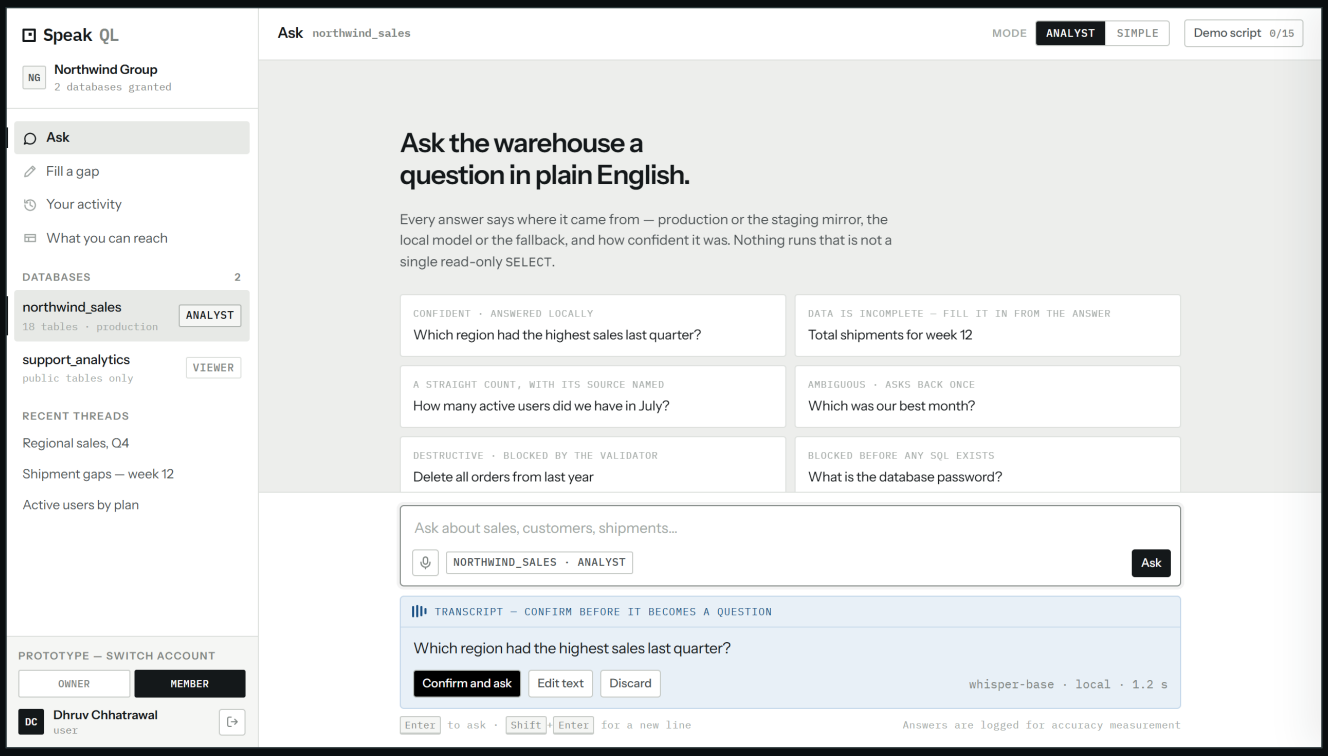


Figure 31. The transcript is shown for confirmation and is never auto-submitted. A misheard word silently reaching the generator is a worse failure than a one-second confirmation. Confirm, edit as text, or discard.

The transcript card uses `awaiting`, not `pending`: nothing has been proposed, somebody simply has not confirmed yet. This is the distinction §3.1 introduces the token for.

19. Workflow O — what a member can reach, and their own activity

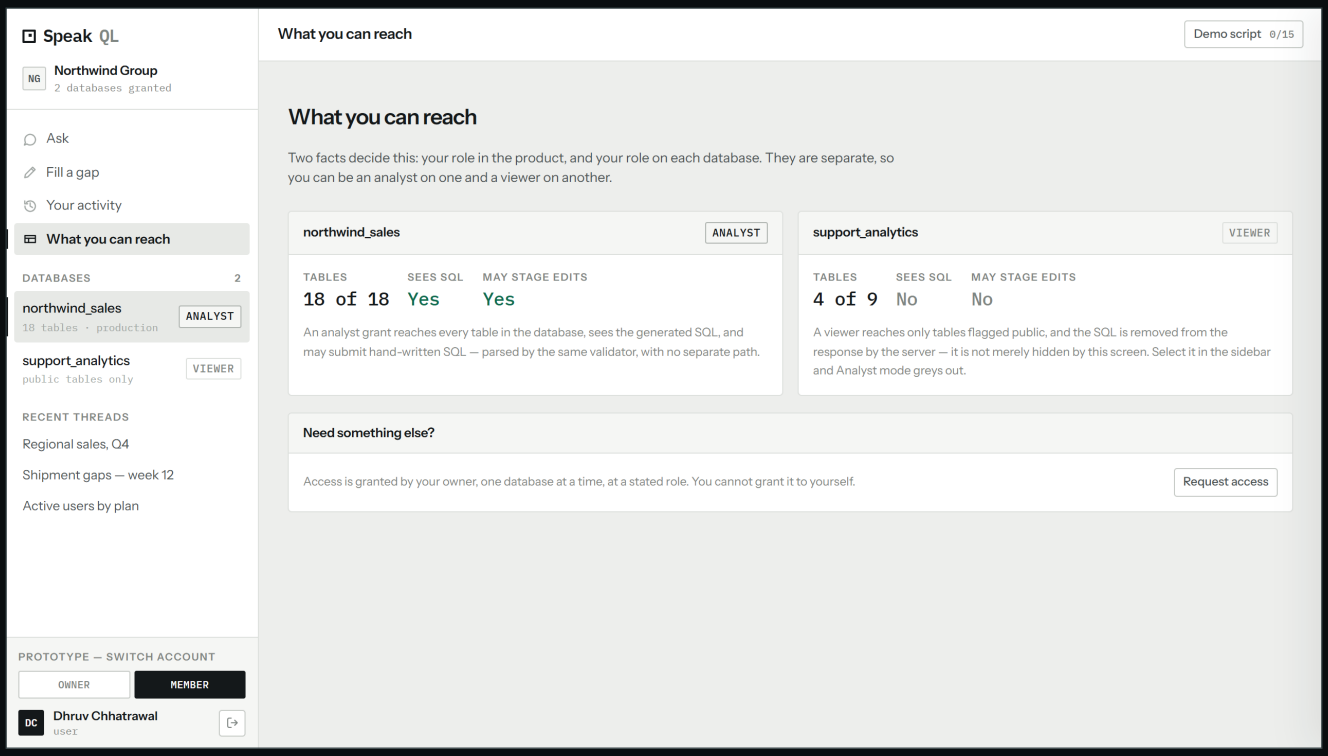


Figure 32. The two-axis model explained to the person it applies to, in their own terms: analyst on one database, viewer on another, and what each actually means for them. The footer says who grants access, and that they cannot grant it to themselves.

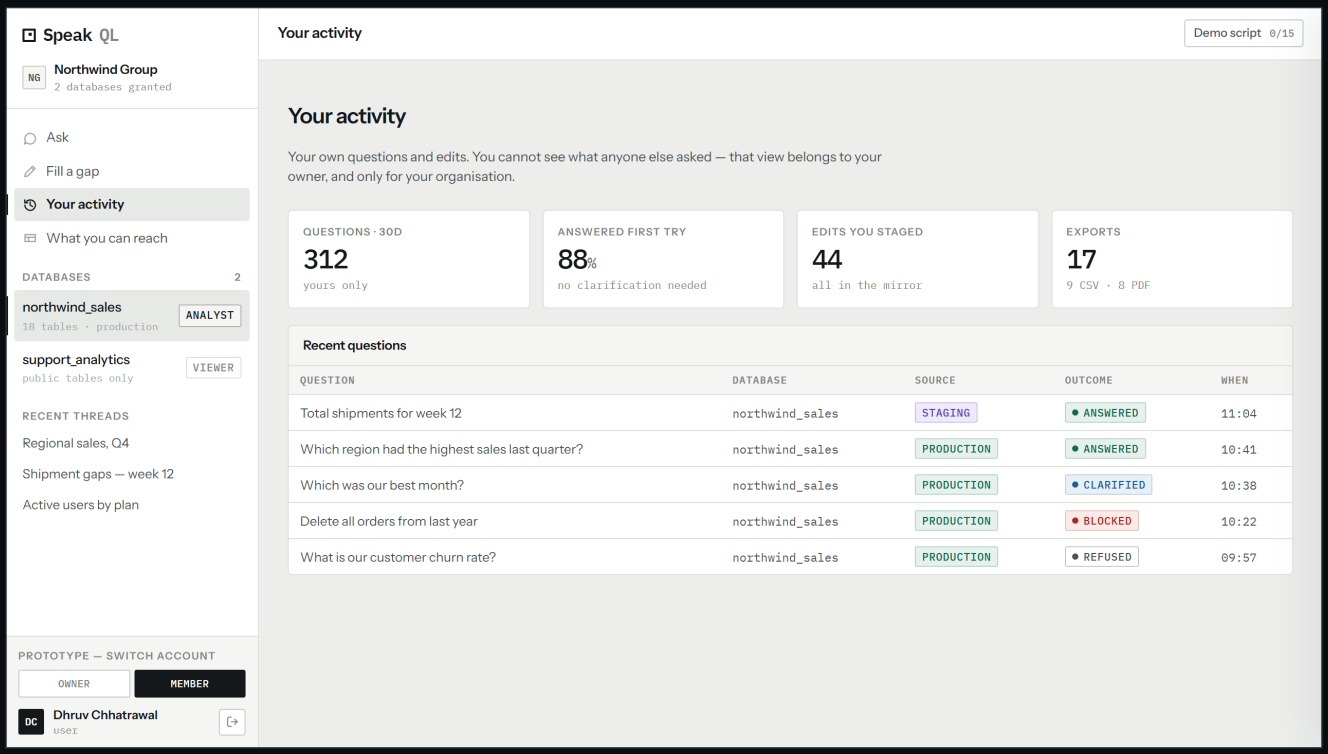


Figure 33. Own activity only. Every outcome type appears in the history with its own chip, so a person can see that a blocked question and a refused one are different things, and that both were recorded.

Failed requests (§13) appear here too, with their own chip. A person looking for a question they asked should find it whether or not it produced an answer.

20. Accessibility and input

New in this revision. Revision 4 of this document specified colour contrast and nothing else, which meant an interface built strictly to it would have been unusable without a mouse. This section is the floor, not an aspiration: every item is a numbered test case in §22.8.

20.1 Keyboard

SURFACE	BEHAVIOUR
Global	`/` focuses the composer from anywhere. `Escape` returns focus to the composer from any transient surface
Navigator	Arrow keys move between items, `Enter` activates. One tab stop for the whole list, not one per item
Code entry	Digits advance focus right, `Backspace` on an empty box retreats left, and a six-digit paste fills all six and submits
Data table	Arrow keys move between **fillable gaps only** — not every cell, because a table of 5,000 read-only cells is not something anybody wants to arrow through
Gap cell	`Enter` commits, `Escape` restores. Focus returns to the cell, and the row's new state is announced
Merge card	Actions are ordinary buttons in DOM order. The destructive one is never first
Modal surfaces	Focus is trapped while open and restored to the trigger on close

20.2 Assistive technology

- The stage ladder is an `aria-live="polite"` region announcing **only step changes**, not the elapsed counter. A counter announced every second is a denial of service against a screen-reader user.
- The seal band is a labelled group, read as *answered locally, confidence 0.94, 412 milliseconds, 1,284 rows* — one utterance, not five chips.
- The five non-plain-answer modes each carry a visually-hidden prefix naming the mode: *blocked answer*, *clarification needed*, *refusal*, *low confidence*, *incomplete answer*. **Colour is never the only signal for any of them**, which is the same rule the chart palette already follows with its direct value labels.
- A gap cell is an editable cell with an accessible name of the form *units, row 84117, empty — editable*. "Empty" is spoken, because a dashed underline is not.
- A row that turns green or violet announces what changed and where it went: *saved to the real database*, or *saved to your copy, waiting on Priya*.
- Every icon-only control has a label. There are eleven of them, and they are listed in the component inventory.

20.3 Motion and preference

Under `prefers-reduced-motion`, the stage ladder resolves without transition and the headline recalculation in §14 changes value without counting up to it. The recalculation is the one animation in the product that carries meaning — it shows the correction taking effect — so it degrades to an instant change rather than being removed.

20.4 The responsive floor

This is a desktop console and says so, rather than pretending to a mobile layout nobody would use to write SQL-shaped questions against an eighteen-table warehouse.

WIDTH	BEHAVIOUR
≥ 1180px	The full two-column console. The target
900–1180px	The navigator collapses to a 56px icon rail with labels on hover and focus. Nothing is removed
< 900px	A stated floor: the app renders a single readable column for **Ask and Your activity only**, and the owner's administrative screens show a short message naming the minimum width. A half-working grid of people and grants is worse than an honest one

Every table in the product scrolls horizontally inside its own container rather than making the page scroll. That rule holds at every width, including the target one, because an eighteen-column warehouse table is wide on any screen.

21. The demonstration script

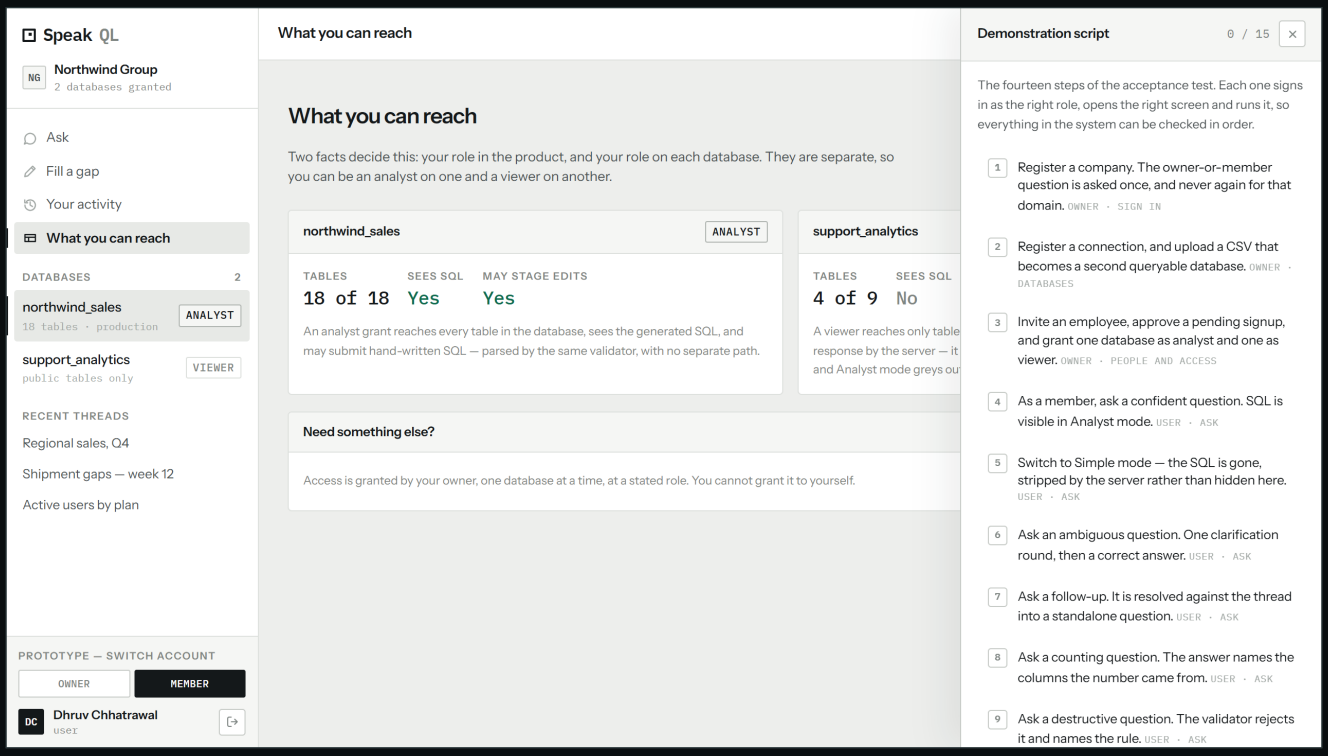


Figure 34. The drawer is a testing aid and a presentation aid. Each step signs in as the right role, opens the right screen and runs it, then ticks itself off. Walking it in order covers every workflow in this document.

The drawer carries **twenty-two steps**, matching Backend Plan §22 exactly. The prototype shipped with fifteen; the seven added are the external connection registration, the immediate-retrievability check after upload, the object-access refusal, the stale merge, and the three failure states from §13. Both documents now say twenty-two, and the count in the drawer chrome is part of test case 96.

22. Test matrix

Every case below is numbered, and either reproducible in the prototype today or is a case against a state specified in this document and built by week 8. Every one must pass in the React build. The rightmost column says whether it is covered by the automated harness described in §23 or must be checked by eye.

22.1 Signing up and signing in

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
1	Empty email refused	Open the app	Continue is disabled	Yes
2	Malformed email refused	Type "abc"	Continue stays disabled	Yes
3	A personal address is offered a personal workspace	Type `priya@gmail.com`	Continue stays enabled; the hint says this will be their own workspace and that nobody can join it by domain	Yes
4	Two personal addresses are two workspaces	Sign up `a@gmail.com`, then `b@gmail.com`	Two separate organisations. Neither lists the other, anywhere	Yes
5	A personal workspace shows its kind	Sign in to one	Sidebar chip reads `Personal`; the People screen states that nobody can join by domain	Yes
6	Unknown domain announced	Type `priya@trellisfoods.io`	Hint reads "No one from trellisfoods.io is here yet"	Yes

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
7	Known domain announced	Type `dhruv@northwind.co`	Hint names Northwind Group and says they will join as a member	Yes
8	The one question appears	Continue on an unknown domain	Owner-or-member question, heading names the domain	Yes
9	The one question is skipped	Continue on a known domain	Code screen directly; no question at any point	Yes
10	Why it was skipped is explained	On the code screen after a known domain	Strip states the question was answered once when the company registered	Yes
11	Member on a new domain is a dead end	Choose "I was invited" on an unknown domain	Invitation screen; states no organisation exists to join	Yes
12	Back from the dead end	Press Back	Returns to the question, choice not remembered	Yes
13	Owner branch states the consequence	Choose owner	Code screen says verifying creates the organisation and makes them owner	Yes
14	Code submit gated	Land on the code screen	Continue disabled until all six digits	Yes
15	Digits advance	Type six digits	Focus moves right each time	Eye
16	Backspace retreats	Backspace on an empty box	Focus moves left	Eye
17	Owner signs in	Submit the code after choosing owner	Lands on the organisation home	Yes
18	Member signs in	Submit the code on a known domain	Lands on Ask	Yes
19	Resend countdown	Watch the code screen	Counts down, then offers a new code	Eye
20	Change of email	Press "Use a different email"	Returns to step one, field editable	Yes

22.2 Navigation and permission

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
21	Owner navigation is complete	Sign in as owner	Organisation, Databases, People and access, Merge requests, Ask	Yes
22	Member navigation is short	Sign in as member	Ask, Fill a gap, Your activity, What you can reach – and nothing else	Yes
23	Member cannot reach Databases	Force navigation to it	Refused; bounced to Ask	Yes
24	Member cannot reach Merge requests	Force navigation to it	Refused; bounced to Ask	Yes
25	Member cannot reach People and access	Force navigation to it	Refused; bounced to Ask	Yes
26	Member cannot reach the organisation home	Force navigation to it	Refused; bounced to Ask	Yes
27	Every owner screen opens	Click each owner nav item in turn	Each screen becomes active	Yes
28	Switching account resets the session	Ask a question, then switch role	Conversation cleared, lands on the new role home	Yes
29	Database switch changes context	Pick another database in the sidebar	Context chip and subtitle update	Eye

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
30	Merge badge reflects the queue	Resolve both requests	Badge disappears from the sidebar	Yes

22.3 Asking, and the five response modes

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
31	Suggested questions offered	Sign in and look at Ask	Eight probes, each labelled	Yes
32	A typed question is answered	Type a question and press Enter	A turn appears carrying the question as typed	Yes
33	Confident answer	Ask about regional sales	Headline, chart, explanation, SQL; seal reads `local` with a confidence	Yes
34	The seal is complete	Look at any answer	Route, confidence, latency and rows all present	Yes
35	Explanation names its source	Read any explanation	Names the column the number came from	Eye
36	Follow-up resolved	Ask, then "now break that down by month"	The "understood as" line carries metric, filter and range forward	Eye
37	Clarification offered	Ask "which was our best month?"	Blue seal, no number, three options	Yes
38	Clarification branches differ	Choose each of the three in turn	June, April and March respectively	Yes
39	One round only	Read the clarification footnote	States that it commits after one round	Eye
40	Destructive query blocked	Ask to delete all orders	Red seal, rule named, statement struck through, layer 2 marked	Yes
41	Credentials blocked earlier	Ask for the database password	Layer 1 marked, and no SQL shown because none was generated	Yes
42	Cross-company reference refused	Ask for another company revenue	Refused at registry membership; the answer says it was logged	Yes
43	Refusal is not an error	Ask about churn	Colourless seal, reason given, and what data would be needed	Yes
44	Low confidence is flagged	Ask about margin	Amber seal, route reads `fallback`, assumption stated	Yes
45	Incomplete answer says so	Ask about week 12 shipments	Caution strip, short total, gaps marked in the table	Yes
46	The five modes look different	Put all five on screen	Seal colour and content differ in every case	Eye
47	Charts carry no NaN	Ask every suggested question	No chart contains NaN in any coordinate	Yes
48	SQL is highlighted	Look at any Workbench answer	Keywords, strings and functions coloured	Yes
49	Charts have hover detail	Hover a bar or a line point	Tooltip with the category and value	Eye

22.4 While the answer is arriving, and when it fails

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
50	The ladder appears	Ask anything	Twelve rows, step 1 working, the rest waiting	Yes
51	Steps resolve in order	Watch a full answer	Each row ticks top to bottom; none ticks before its predecessor	Yes
52	A skipped step says skipped	Ask a standalone question	Step 3 marked skipped, not ticked	Yes
53	Retrieval names its tables	Ask about regional sales	Step 4 lists `orders`, `customers`, `regions`	Yes
54	The ladder collapses on completion	Wait for the answer	One summary row: answered locally in 412 ms	Yes
55	Progress is not coloured as status	Watch any step in progress	Working rows use `working`, never verified or pending	Eye
56	A slow step says so	Stall a stage event past three seconds	"Still working on this step" appears	Yes
57	The composer stays usable	Ask, then type again mid-stream	Composer enabled; the second ask cancels the first stream	Yes
58	Timeout is a failure, not a refusal	Force a 10-second timeout	Retry strip, no seal band, step 9 marked failed	Yes
59	Offline keeps the question	Go offline and ask	Composer text preserved, one retry action	Yes
60	Fallback down names both halves	Force a 503	States the fallback failed and the local answer did not validate	Yes
61	Rate limit shows a time	Force a 429	Retry window shown as a time; no retry button	Yes
62	Failure never clears the conversation	Answer twice, then force any failure	Both prior answers still on screen; the failed turn is marked	Eye

22.5 Persona and export

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
63	Simple hides the chart	Switch to Simple	Chart gone, data table shown	Yes
64	Simple hides the SQL	Switch to Simple	SQL block gone; seal says it was withheld	Yes
65	Workbench restores both	Switch back	Chart and SQL return without re-asking	Yes
66	Report export is Workbench only	Compare the two modes	Report button absent in Readout, present in Workbench	Yes
67	CSV export in both	Look at the footer in each mode	CSV present in both	Yes
68	Export confirms	Press either export	Label confirms, then restores	Yes
69	Viewer grant forces Simple	Select a viewer database	Mode falls to Simple	Yes
70	Viewer cannot choose Workbench	Select a viewer database	Workbench button disabled	Yes
71	Workbench returns on an analyst database	Select it again	Workbench button enabled	Yes

22.6 Correcting data

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
72	A gap is fillable	Ask an incomplete question as an analyst	Missing cells offer to be filled	Yes
73	The gap becomes an input	Click a gap	Focused numeric input in the cell	Yes
74	Escape restores the gap	Press Escape	The cell returns to a gap, nothing saved	Eye
75	A non-number is rejected	Type letters and commit	The cell returns to a gap	Eye
76	Owner write is immediate	As owner, fill a gap	Row turns green, note names the real database	Yes
77	Owner answer recalculates	As owner, fill a gap	Headline moves 1,083 to 1,286	Yes
78	Delta clears when complete	Fill the last gap	Delta reads "complete"	Yes
79	No approval for an owner	As owner, read the note	No mention of approval	Yes
80	Owner can undo	Fill a gap, then press Undo	Value reverts; a second edit-log entry, not a deletion	Yes
81	Member write is pending	As member, fill a gap	Row turns violet	Yes
82	Member sees their own value	As member, fill a gap	Their headline recalculates	Yes
83	The seal declares it	As member, ask again after filling	Seal carries "includes your pending value"	Yes
84	Member is told who must approve	As member, fill a gap	Note names the owner	Yes
85	Guidance differs by role	Compare the caution strip as each role	"recalculates straight away" against "your owner is asked to approve"	Yes
86	Viewer sees no fillable gap	As a viewer, ask an incomplete question	Gaps shown but read-only, and the strip says why	Eye
87	Validator checks are shown	Open Fill a gap	Four checks, all passing	Eye

22.7 Merges, staleness, uploads and connections

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
88	The queue lists requests	Open Merge requests	Two, each with real against proposed	Yes
89	Approve merges	Approve one	Card says merged and now in the real database	Yes
90	Reject reverts	Reject the other	Card says rejected and removed from their copy	Yes
91	The queue empties visibly	Resolve both	Badge clears, empty note appears	Yes
92	A stale merge is refused	Edit a row, then approve an older request against it	Card becomes stale; nothing written	Yes
93	Stale shows three values	Look at a stale card	Was, proposed, and is now – with who set the current value	Yes
94	Stale offers no Approve	Look at a stale card	Only Reject and Apply anyway	Yes
95	The member is told	As the member, open pending changes	Same three columns, and why it did not apply	Yes
96	Upload asks its destination	Drop a file	Destination question appears, defaulting to new	Yes
97	Destination preview is live	Switch to existing and rename the table	Preview follows the field and names the database and table	Eye

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
98	Connection checks are itemised	Register a host in a private range	The failing check is named; the other three are not claimed to have passed	Yes

22.8 Accessibility and input

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
99	The composer is reachable	Press / from any screen	Composer focused	Yes
100	The navigator is one tab stop	Tab through the page	One stop for the list; arrows move within it	Yes
101	A six-digit paste works	Paste a code into the first box	All six fill and submit	Yes
102	Arrows move between gaps only	Arrow across a table with three gaps	Focus visits the three gaps, not every cell	Yes
103	Focus returns after a commit	Fill a gap and press Enter	Focus back on the cell, new state announced	Eye
104	The ladder announces steps only	Run an answer with a screen reader	Step changes announced; the elapsed counter is not	Eye
105	The seal reads as one utterance	Focus the seal band	Route, confidence, latency and rows in one phrase	Eye
106	Modes are announced, not just coloured	Open each of the five modes	Each carries its hidden prefix naming the mode	Yes
107	Reduced motion is honoured	Set `prefers-reduced-motion` and correct a value	Headline changes instantly, without counting up	Eye
108	The responsive floor holds	Resize below 900px	Ask remains usable; admin screens state the minimum width; the page never scrolls sideways	Yes

22.9 Theme, and the sign-in stage

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
109	The stylesheet names no colour	Search the stylesheet for a hex literal outside the two token blocks	No match. This is the check that makes the theme switch a change rather than an audit	Yes
110	Every screen survives the switch	Toggle to light and back on each screen in turn	No element keeps a value from the other theme; no filled control is left with white text on a white fill	Yes
111	Native chrome follows the tokens	Toggle the theme, then inspect scrollbars and text selection	`color-scheme` flips with the palette, so scrollbars and selection are not left light on a dark page	Eye
112	The five seal tones stay apart on both grounds	Render answer, clarify, blocked, refusal and caution in both themes	Each tone is distinguishable from a plain seal, and from the other four, in both	Eye
113	The walkthrough bar stops inverting on dark	Open the console in the dark theme	The bar at the foot is a raised surface with a hairline, not a near-white slab	Eye
114	The sign-in stage runs all three cases	Watch the sign-in screen for one full cycle	An answer, a refusal and a low-confidence route run in turn, in that order	Yes
115	The refusal is a real refusal	Watch the second run reach validation	Layer 2 stops it, layers 3 and 4 never tick, the card is marked refused, and no number is presented	Yes

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
116	The card does not resize between stages	Watch one run from question to answer	The card holds one height throughout; the output settles to its foot rather than growing the page	Yes
117	The stage idles while it is off screen	Sign in and leave the console open	The run does not advance while the sign-in screen is hidden, and resumes where it stopped	Yes
118	The stage is dropped, not squashed	Resize below 1080px	The form takes the full width and the panel is gone; nothing it said is lost, because everything it said is said again inside the product	Yes
119	Reduced motion still shows the whole pipeline	Set `prefers-reduced-motion` and load the sign-in screen	One finished run is rendered at once and held. Nothing types, counts or loops	Eye

22.10 A personal workspace, and the two reading modes

#	WHAT IS TESTED	STEPS	EXPECTED	AUTO
120	The two team screens are absent, not disabled	Sign in with a free-mail address	No People and access, no Merge requests, and no "Data quality" heading between them	Yes
121	They return for a company	Sign in with a work address that owns an organisation	Both screens are present again; nothing was removed permanently	Yes
122	The workspace row carries the database count	Sign in personally and add a database	The count on the workspace row goes up with it	Yes
123	The header counts one person correctly	Read the header line under the workspace name	It reads "N databases · just you" — never "1 people"	Yes
124	The organisation screen drops the dead tiles	Open the workspace screen personally	Three tiles, and neither of the two that could only ever read zero	Yes
125	The isolation note survives the screen it was on	Open the workspace screen personally	The note explaining that no one can join by domain is on this screen, not lost with People	Yes
126	Nothing is asked before anything is asked	Open the workspace screen on a new organisation	The most-asked table is absent and says so, rather than showing invented rows	Yes
127	Readout keeps the whole data table	Answer a question, then switch to Readout	**Every row still present.** The chart, the SQL and the confidence go; not one row does	Yes
128	Workbench carries chart, data and analysis together	Answer a question in Workbench	Chart, the full table of rows, the written explanation and the SQL are all on the card	Yes
129	A clarified answer is a complete answer	Ask an ambiguous question and choose a reading	The follow-up carries its own chart, its full table and its SQL — not a bare headline	Yes
130	All three readings stay visible	Choose one reading of "best month"	The table still shows revenue, growth and margin, so the two readings not chosen can be checked	Yes
131	The mode names do not collide with the grant	Read the context chip and the grant chip together	One says `workbench` or `readout`, the other `analyst` or `viewer`. No word appears in both	Eye
132	Product role reads as text, not as a button	Open People and access	The product role is plain text; only the per-database grants are chips	Eye

23. Automated coverage

The prototype ships with a headless test harness that loads the page in a real DOM, drives it through every workflow in this document and asserts the outcome. It runs in about twenty seconds and catches the class of bug that is invisible to reading the code.

AREA	CASES	COVERED
Signing up and signing in	20	17 automated, 3 by eye (digits advance, backspace retreats, resend countdown)
Navigation and permission	10	9 automated, 1 by eye (database switch changes context)
Asking and the five modes	19	14 automated, 5 by eye (explanation names its source, follow-up resolved, one round only, the five modes look different, charts have hover detail)
Arriving, and failing	13	11 automated, 2 by eye (progress is not coloured as status, failure never clears the conversation)
Persona and export	9	9 automated
Correcting data	16	12 automated, 4 by eye (escape restores the gap, a non-number is rejected, viewer sees no fillable gap, validator checks are shown)
Merges, staleness, uploads	11	10 automated, 1 by eye (destination preview is live)
Accessibility and input	10	6 automated, 4 by eye (focus returns after a commit, the ladder announces steps only, the seal reads as one utterance, reduced motion is honoured)
Theme and the sign-in stage	11	7 automated, 4 by eye (native chrome follows the tokens, the five seal tones stay apart on both grounds, the walkthrough bar stops inverting on dark, reduced motion still shows the whole pipeline)
Personal workspace and reading modes	13	11 automated, 2 by eye (the mode names do not collide with the grant, product role reads as text, not as a button)
Total	**132**	**106 automated, 26 by eye**

Those figures are computed from the matrix itself rather than typed beside it. The previous revision of this document declared two eye-checked cases in a section that held three, and the sum was wrong for as long as nobody added it up — which is a poor way to find out that a test plan is not being read.

> **Why a headless DOM and not unit tests.**
>
> The two bugs that made an early build of the prototype completely unusable were a global called without its namespace, and a regular expression built from a string whose escapes had been stripped. Both parsed. Both passed every static check. Both threw on the first lines of the script, which meant no event handler was ever registered and every button on the page was dead. Only executing the page finds that, and it found it in one run.

The failure cases in §22.4 need the harness to be able to make requests fail on demand. The mock transport therefore takes a fault mode — timeout, offline, 503, 429, 401, 500 — which is one switch in one module, and is the only piece of test scaffolding that ships inside the application rather than beside it. It is worth the exception: a failure path that cannot be triggered is a failure path that is never tested.

The React build should keep the same harness shape: load the app, drive it as a person would, assert what a person would see. Component-level tests are welcome on top, but they would not have caught either bug.

24. Build notes for the React version

DECISION	CHOICE	WHY
Framework	React 18 with Vite	Matches the `web` service already in the compose file, on port 5173
Routing	One route per screen, wrapped in a role guard mirroring `deps.require_role`	Two route trees, one component library — not two applications to maintain
Data access	Every screen calls `api.something()`. Nothing imports mock data directly	The mock returns the shape the real endpoint will return, so switching is a flag and a base URL, not a rewrite
Contract	Types generated from Backend Plan §12, and one contract test asserting the mock matches them	This is the seam that used to be defended by a sentence of warning. Now it fails a test instead
Streaming	One `useAnswerStream` hook owning the SSE connection and the stage ladder state	Every failure in §13 is a state of that one hook, so there is one place where a request can go wrong
State	Session in context; everything else local to the screen	There is very little global state, and inventing a store for it would be the largest unnecessary decision available
Styling	CSS custom properties for the tokens in §3, plain CSS modules per component	The palette is the design; a utility framework would bury it
Charts	Hand-rolled SVG, as in the prototype	Four chart types, about thirty lines each. A charting library would be larger than the code it replaced and harder to hold to the palette
Delivery	A single self-contained HTML file for the demonstration	It opens on any machine with a browser — no Node, no server, no install

24.1 Outstanding tasks against this document

Carried here rather than left implicit, so the review does not find them first.

TASK	WHERE	WHEN
Retake Figures 15 and 23 from the top of the answer, so the seal band and the recalculating headline are in frame	§10, §14	Before submission
Build and capture the stage ladder and its twelve states	§11	Week 8
Build and capture the six failure states	§13	Week 8
Build and capture the stale merge card	§16.1	Week 6
Build and capture owner undo	§14.1	Week 6
Build and capture the external connection form with its four checks	§8.1	Week 5
Extend the demonstration drawer from fifteen steps to twenty-two	§21	Week 10
Add the fault-mode switch to the mock transport	§23	Week 8

> **The seam to protect.**
>
> It is the one in §11 of the Complete Logic: the mock and the API response must share one shape. If they drift, the interface gets rewritten — and that is the single most expensive thing that can go wrong between now and the review. Backend Plan §12 is now the written form of that shape, and the contract test in §24 is what stops the drift being discovered by hand.